



Welcome! In this course, we are going to create a complete battle system for a turn-based monster-fighting game. You will build a system where the player can engage in battles with trainers or encounter wild monsters, which can also be caught. This is an advanced Godot tutorial that will teach you a wide range of concepts, from state management to working with complex data and UI systems.

Course Structure

This course is the second part of a three-part series. The first part focused on creating the game's overworld. However, the overworld and the battle system are designed to be modular. Therefore, if you are only interested in creating a battle system, you can follow this course in isolation. If you have already completed the first part, you can simply continue with your existing project. In the third and final part, we will combine the overworld and the battle system into a single, cohesive game.

What You Will Create

The main focus of this course is the battle system. By the end, you will have built the following features:

- A complete turn-based system where the player can attack.
- The ability for the player to swap between different monsters in their party.
- A mechanic for catching wild monsters.
- A monster object that effectively combines 2D nodes with Control (UI) nodes.

Key Learning Concepts

While building this complex project, you will explore several advanced Godot functionalities. These types of games rely heavily on accessing and managing data, which can be a challenging but rewarding task. The main goals are:

- **State Management with Enums:** We will use enums to manage the various states of the battle menu. This allows us to track whether the player is in the process of attacking, swapping a monster, or catching one.
- **Data Management with Resources:** You will learn how to use Godot's powerful Resource system to create, store, and manage all the data for your monsters.
- **Combining 2D and Control Nodes:** This course will guide you through the process of integrating 2D game elements with UI components, a technique that can sometimes be tricky.
- **Complex Project Logic:** Be prepared to write a significant amount of code to manage the game's logic. This project will provide excellent practice in structuring a complex system.

Prerequisites

This is an advanced course, and it is expected that you are already familiar with how Godot works. Ideally, you should have already created a few smaller games and are comfortable with the engine's core features.

- You should be familiar with the Godot engine and its interface.
- You are expected to have a basic understanding of concepts such as enums and tweens.
- While some advanced topics like Resources will be explained in detail, more fundamental concepts will be taken for granted.

Now that you know what to expect, let's begin the course and start building our battle system!

Which version of Godot should you use for this course?

Technology changes rapidly, so to make sure you have an optimal learning experience, we recommend using **Godot 4.5** for this course – the latest stable release.

Please make sure NOT to use newer versions of the software than what we recommend, as the course material provided might not work as expected.

How to Install Version 4.5

You can download the most recent version of Godot by heading to the Godot website (<https://godotengine.org/>) and clicking the “Download Latest” button on the front page. This will automatically take you to the download page for your operating system.



Once on the download page, just click the option for **Godot 4.5**. This will download the Godot engine to your local computer. From there, unzip the file and simply click on the application launcher – no further installation steps are required!

Download Godot 4 for Windows

**Godot Engine****4.5**

x86_64 - 15 September 2025

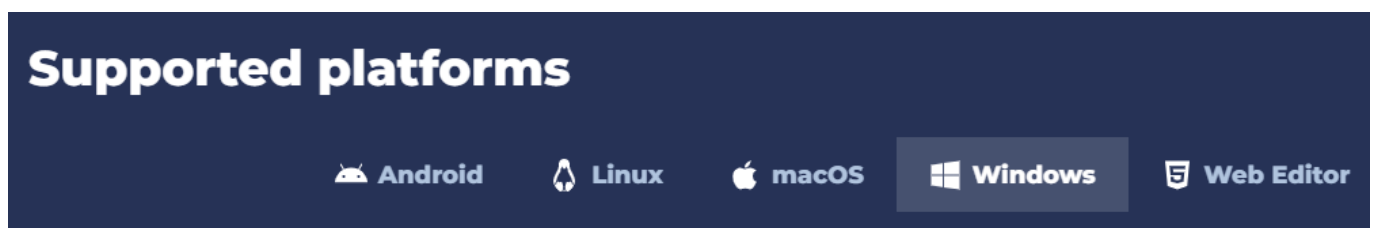
**Godot Engine – .NET****4.5**

x86_64 · C# support · 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?



If Godot fails to automatically select the correct operating system for you, you can scroll down to the “Supported platforms” section on the download page to manually select the download version you would like to install:





Before we begin developing our monster-battling game, it is crucial to understand the initial project setup. This project already contains numerous scenes, code, and assets that will serve as the foundation for our work. While there is a complete overworld system from a previous course, we will not be interacting with it. Our focus will be exclusively on the data structures and scenes required to build the battle system. All necessary files are available in the course resources.

The Existing Project Structure

When you open the project in Godot and run it, the main scene that loads is `overworld.tscn`. This scene includes:

- A tilemap layer for the background environment.
- Various characters, including the player and trainers.
- Simple objects like buildings and arenas.
- Transition gates that allow movement between different scenes.

Again, we will not be modifying any of these overworld elements. The only file from the initial setup that we need to be familiar with is `data.gd`.

The Core Data File: `data.gd`

The most important file for our battle system is `data.gd`. This script is configured as a global script (an “AutoLoad” in Godot), which means its data is accessible from anywhere in the project. This file acts as a central database for the game. While it contains some data for the overworld, we are only concerned with the **battle data** section.

```
# battle data
enum Monster {SPARCHU, ATROX, JACANA, CINDRILL, CLEAF, DRAEM, FINIETTE, FINSTA, FRIOL
ERA, GULFIN, IVIERON, LARVEA, PLUMA, PLUMETTE, POUCH, VULKEO}
enum Element {FIRE, WATER, PLANT}
enum Attack {CLAW, FIRE, ICE, WATER, HEAL, EXPLOSION}
enum MenuState {MAIN, ATTACK, DEFEND, SWAP, CATCH, SELECT}
enum Biome {GRASS, DESERT, ICE}
const monster_data = {
    Monster.SPARCHU: {
        'name': 'Sparchu',
        'battle texture': "res://graphics/battle sprites/Sparchu.png",
        'icon texture': "res://graphics/menu sprites/Sparchu.png",
        'stats': {'max hp': 10, 'max ep': 2, 'speed': 1},
        'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION},
        'element': Element.FIRE},
    Monster.CINDRILL: {
        'name': 'Cindrill',
        'battle texture': "res://graphics/battle sprites/Cindrill.png",
        'icon texture': "res://graphics/menu sprites/Cindrill.png",
        'stats': {'max hp': 14, 'max ep': 12, 'speed': 1},
        'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION},
        'element': Element.FIRE},
    Monster.VULKEO: {
        'name': 'Vulkeo',
        'battle texture': "res://graphics/battle sprites/Vulkeo.png",
        'icon texture': "res://graphics/menu sprites/Vulkeo.png",
        'stats': {'max hp': 17, 'max ep': 14, 'speed': 1},
```

```
'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION},
'element': Element.FIRE},
Monster.FINSTA: {
'name': 'Finsta',
'battle texture': "res://graphics/battle sprites/Finsta.png",
'icon texture': "res://graphics/menu sprites/Finsta.png",
'stats': {'max hp': 5, 'max ep': 6, 'speed': 0.7},
'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
'element': Element.WATER},
Monster.GULFIN: {
'name': 'Gulfin',
'battle texture': "res://graphics/battle sprites/Gulfin.png",
'icon texture': "res://graphics/menu sprites/Gulfin.png",
'stats': {'max hp': 9, 'max ep': 8, 'speed': 0.8},
'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
'element': Element.WATER},
Monster.FINIETTE: {
'name': 'Finiette',
'battle texture': "res://graphics/battle sprites/Finiette.png",
'icon texture': "res://graphics/menu sprites/Finiette.png",
'stats': {'max hp': 11, 'max ep': 10, 'speed': 0.9},
'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
'element': Element.WATER},
Monster.PLUMETTE: {
'name': 'Plumette',
'battle texture': "res://graphics/battle sprites/Plumette.png",
'icon texture': "res://graphics/menu sprites/Plumette.png",
'stats': {'max hp': 6, 'max ep': 7, 'speed': 0.6},
'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
'element': Element.PLANT},
Monster.IVIERON: {
'name': 'Pluma',
'battle texture': "res://graphics/battle sprites/Pluma.png",
'icon texture': "res://graphics/menu sprites/Pluma.png",
'stats': {'max hp': 8, 'max ep': 8, 'speed': 0.9},
'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION},
'element': Element.PLANT},
Monster.PLUMA: {
'name': 'Pluma',
'battle texture': "res://graphics/battle sprites/Pluma.png",
'icon texture': "res://graphics/menu sprites/Pluma.png",
'stats': {'max hp': 12, 'max ep': 9, 'speed': 1.1},
'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION},
'element': Element.PLANT},
Monster.LARVEA: {
'name': 'Larvea',
'battle texture': "res://graphics/battle sprites/Larvea.png",
'icon texture': "res://graphics/menu sprites/Larvea.png",
'stats': {'max hp': 6, 'max ep': 7, 'speed': 0.6},
'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION},
'element': Element.PLANT},
```



```
Monster.CLEAF: {
  'name': 'Pluma',
  'battle texture': "res://graphics/battle sprites/Pluma.png",
  'icon texture': "res://graphics/menu sprites/Pluma.png",
  'stats': {'max hp': 9, 'max ep': 8, 'speed': 1},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSI
ON},
  'element': Element.PLANT},
Monster.DRAEM: {
  'name': 'Draem',
  'battle texture': "res://graphics/battle sprites/Draem.png",
  'icon texture': "res://graphics/menu sprites/Draem.png",
  'stats': {'max hp': 12, 'max ep': 9, 'speed': 1.1},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSI
ON},
  'element': Element.PLANT},
Monster.POUCH: {
  'name': 'Pouch',
  'battle texture': "res://graphics/battle sprites/Pouch.png",
  'icon texture': "res://graphics/menu sprites/Pouch.png",
  'stats': {'max hp': 14, 'max ep': 3, 'speed': 0.5},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSI
ON},
  'element': Element.PLANT},
Monster.ATROX: {
  'name': 'Atrox',
  'battle texture': "res://graphics/battle sprites/Atrox.png",
  'icon texture': "res://graphics/menu sprites/Atrox.png",
  'stats': {'max hp': 12, 'max ep': 8, 'speed': 2},
  'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSI
N},
  'element': Element.FIRE},
Monster.FRIOLERA: {
  'name': 'Friolera',
  'battle texture': "res://graphics/battle sprites/Friolera.png",
  'icon texture': "res://graphics/menu sprites/Friolera.png",
  'stats': {'max hp': 11, 'max ep': 10, 'speed': 0.9},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
  'element': Element.WATER},
Monster.JACANA: {
  'name': 'Jacana',
  'battle texture': "res://graphics/battle sprites/Jacana.png",
  'icon texture': "res://graphics/menu sprites/Jacana.png",
  'stats': {'max hp': 3, 'max ep': 5, 'speed': 4},
  'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSI
N},
  'element': Element.FIRE}}
const attack_data = {
  Attack.CLAW: {
    'name': 'Claw',
    'offensive': true,
    'cost': 50,
    'amount': 10,
    'element': Element.PLANT,
    'texture': "res://graphics/attack effects/attack7.png",
```



```
'sound': "res://audio/scratch.mp3"},
Attack.FIRE: {
  'name': 'Fire',
  'offensive': true,
  'cost': 12,
  'amount': 15,
  'element': Element.FIRE,
  'texture': "res://graphics/attack effects/attack5.png",
  'sound': "res://audio/fire.wav"},
Attack.WATER: {
  'name': 'Splash',
  'offensive': true,
  'cost': 9,
  'amount': 11,
  'element': Element.WATER,
  'texture': "res://graphics/attack effects/attack3.png",
  'sound': "res://audio/splash.wav"},
Attack.ICE: {
  'name': 'Ice shards',
  'offensive': true,
  'cost': 13,
  'amount': 20,
  'element': Element.WATER,
  'texture': "res://graphics/attack effects/attack1.png",
  'sound': "res://audio/ice.mp3"},
Attack.EXPLOSION: {
  'name': 'Explode',
  'offensive': true,
  'cost': 20,
  'amount': 100,
  'element': Element.FIRE,
  'texture': "res://graphics/attack effects/attack4.png",
  'sound': "res://audio/explosion.mp3"},
Attack.HEAL: {
  'name': 'Heal',
  'offensive': false,
  'cost': 15,
  'amount': -30,
  'element': Element.PLANT,
  'texture': "res://graphics/attack effects/attack6.png",
  'sound': "res://audio/green.wav"}}
const element_modifier = {
  Element.FIRE: {Element.FIRE: 1, Element.WATER: 0.5, Element.PLANT: 2},
  Element.WATER: {Element.FIRE: 2, Element.WATER: 1, Element.PLANT: 0.5},
  Element.PLANT: {Element.FIRE: 0.5, Element.WATER: 2, Element.PLANT: 1}}
```

Organizing Data with Enums

To keep the code clean and readable, the script uses several enums. An enum (enumeration) is a way to assign descriptive names to a set of integer constants. This makes the code easier to understand and maintain. The enums defined for our battle system are:

- **Monster:** A list of all available monsters (e.g., Spudger, Aatrox).



- **Elements:** The different elemental types (e.g., Fire, Water, Plant).
- **AvailableAttacks:** A list of all possible attacks in the game.
- **MenuStates:** Tracks the player's current action in battle (e.g., Attacking, Defending, Swapping).
- **Biomes:** Defines the different types of environments (e.g., Grass, Desert, Ice).

Storing Complex Information with Dictionaries

For more detailed information, the project uses dictionaries. A dictionary stores data in key-value pairs, which is perfect for defining the properties of our monsters and their attacks. There are three primary dictionaries we will use:

- **ELEMENT_MODIFIER:** This dictionary tracks elemental strengths and weaknesses. For example, it defines that a Fire attack deals double damage to a Plant monster but only half damage to a Water monster.
- **MONSTER_DATA:** This is a large dictionary where each key is a monster from the Monster enum. The value for each monster contains its specific attributes:
 - Name
 - Battle and icon textures
 - Stats (Health, Energy, Speed)
 - A list of available attacks and the level at which they are learned
 - The monster's primary element
- **ATTACK_DATA:** Similar to the monster data, this dictionary defines the properties for every attack listed in the AvailableAttacks enum. Each entry includes:
 - Name
 - Type (Offensive or Defensive)
 - Energy cost
 - Damage amount
 - The attack's element
 - The texture and sound for its animation

Important Project Settings

Finally, a few project settings have been pre-configured to ensure the game looks and feels correct. You can view these under *Project > Project Settings*.

1. **Window Size:** The viewport resolution is set to 1280×720 pixels.
2. **Texture Filtering:** Under *Rendering > Textures*, the Default Texture Filter is set to "Nearest". This is essential for pixel art games as it ensures the graphics remain sharp and crisp without any blurring.
3. **Input Map:** Basic player inputs have already been defined in the Input Map tab.

With this foundational understanding of the project's data and settings, we are now prepared to begin building the battle system. In the upcoming lessons, we will leverage these data structures to bring our monster battles to life.



Welcome to the lesson on setting up our battle scene. In this article, we will create the foundational elements for our monster battles. This includes preparing the background, defining default positions for the monsters, and creating a visual representation for them, complete with animations. By the end of this lesson, you will have a basic but functional battle stage ready for the logic we will add in subsequent parts.

Creating the Battle Scene

First, we need a dedicated scene for our battles. This will serve as the container for all battle-related elements.

1. Inside Godot, create a new 2D scene.
2. Rename the root node from Node2D to Battle.
3. Save this scene. Create a new folder named battle inside the scenes directory and save the scene as battle.tscn.

To ensure this scene runs when we launch the game, we must set it as the main scene.

1. Navigate to **Project > Project Settings**.
2. Go to the **Application > Run** tab.
3. For the **Main Scene** property, click the folder icon and select your newly created scenes/battle/battle.tscn file.

If you run the project now, you will see a blank screen, which is the correct starting point.

Setting Up the Background

A battle scene needs a backdrop. We will add a background image and a camera to ensure it fills the entire screen.

1. Add a Sprite2D node as a child of the Battle node and rename it to BG.
2. In the Inspector for the BG node, locate the **Texture** property. Drag one of the background images from graphics/backgrounds/ (e.g., the forest background) into this property.
3. To make the background fill the view, add a Camera2D node as a child of the BG node.
4. Select the Camera2D node. In the Inspector, find the **Zoom** property and set its X and Y values to 2. This should scale the view so the background image fits perfectly.
5. To prevent accidentally moving the background, select the BG node and click the lock icon next to its name in the Scene dock.

Running the scene now will display the background image covering the entire game window.

Defining Start Positions

Next, we need to define where the player's and the enemy's monsters will appear. We will use Marker2D nodes for this, as they are perfect for storing positions.

1. Add a Node2D as a child of Battle and name it StartPositions.
2. Add two more Node2D children to StartPositions, naming them Player and Enemy.
3. Under the Player node, add three Marker2D nodes. Position them on the left side of the screen: one towards the top, one in the middle, and one at the bottom.
4. Duplicate these three markers and move them under the Enemy node. Position them similarly, but on the right side of the screen.

These markers will serve as the spawn points for our monsters.



Creating Monster Containers

We also need parent nodes that will hold the actual monster sprites once they are instantiated in the battle.

1. Add a Node2D as a child of Battle and name it BattleSprites.
2. Add two Node2D children to BattleSprites, naming them Player and Enemy.

Your final scene tree for the Battle scene should look like this:

- Battle (Node2D)
 - BG (Sprite2D)
 - Camera2D
 - StartPositions (Node2D)
 - Player (Node2D)
 - Marker2D
 - Marker2D2
 - Marker2D3
 - Enemy (Node2D)
 - Marker2D
 - Marker2D2
 - Marker2D3
 - BattleSprites (Node2D)
 - Player (Node2D)
 - Enemy (Node2D)

The BattleSprite Scene and Animations

Now, let's create a separate scene for the monster's visual representation. This scene will handle the monster's appearance and animations.

1. Create a new 2D scene with a Sprite2D as the root node. Rename it to BattleSprite.
2. Save the scene as battle_sprite.tscn inside the scenes/battle folder.
3. Select the BattleSprite node. For its **Texture**, choose any monster sprite from the graphics/battle sprites/ folder (e.g., Sparchu.png).
4. In the Inspector, under the **Animation** section, set **Hframes** to 4 and **Vframes** to 2. This divides the sprite sheet into a 4×2 grid, matching our animation frames.
5. Add an AnimationPlayer node as a child of BattleSprite.

We will now create two animations: idle and attack.

- **Idle Animation:**

1. In the AnimationPlayer panel, create a new animation named idle.
2. Set its length to 0.8 seconds and enable looping and autoplay.
3. Add a property track for the BattleSprite node's frame property.
4. Add four keyframes at times 0.0, 0.2, 0.4, and 0.6, with frame values of 0, 1, 2, and 3, respectively.

- **Attack Animation:**

1. Create another new animation named attack.
2. Set its length to 0.8 seconds.
3. Add a property track for the BattleSprite node's frame property.
4. Add four keyframes at times 0.0, 0.2, 0.4, and 0.6, with frame values of 4, 5, 6, and 7, respectively.

If you play the idle animation, you should see your monster animate smoothly.



Spawning Monsters with Code

Finally, let's write the code to spawn our monsters at the start positions we defined.

Battle Script (battle.gd)

Attach a new script to the root Battle node.

```
extends Node2D

var battle_sprite_scene = preload("res://scenes/battle/battle_sprite.tscn")

func _ready():
    battle_sprite_setup($StartPositions/Player, $BattleSprites/Player, true)
    battle_sprite_setup($StartPositions/Enemy, $BattleSprites/Enemy, false)

func battle_sprite_setup(start_positions: Node2D, parent: Node2D, is_player: bool):
    for start_pos_marker : Marker2D in start_positions.get_children():
        var pos = start_pos_marker.position
        create_battle_sprite(pos, parent, is_player)

func create_battle_sprite(pos: Vector2, parent: Node2D, is_player: bool):
    var battle_sprite = battle_sprite_scene.instantiate()
    battle_sprite.setup(pos, is_player)
    parent.add_child(battle_sprite)
```

This script preloads our BattleSprite scene. In the `_ready` function, it calls `battle_sprite_setup` for both the player and the enemy. This function iterates through the `Marker2D` positions and calls `create_battle_sprite` for each one, which instantiates, parents, and sets up the monster sprite.

BattleSprite Script (battle_sprite.gd)

Now, attach a script to the root BattleSprite node in the `battle_sprite.tscn` scene.

```
extends Sprite2D

func setup(pos: Vector2, is_player: bool):
    position = pos
    flip_h = is_player
```

This script defines the `setup` function called from our `battle.gd` script. It sets the monster's position and, crucially, flips the sprite horizontally if it belongs to the player (`is_player` is true). This makes the player's monsters face the enemy monsters.

If you run the main scene now, you should see six monsters appear on the screen—three on the left facing right, and three on the right facing left. This provides a solid visual foundation for the battle system we will continue to build.



In this lesson, we will focus on creating a data structure for our monsters. This data will not be directly visible in the game; instead, it will serve as a container to keep track of each monster's individual state. The visual representation will be handled by the battle sprite, which we will connect to this data structure.

Tracking Monster Data

For each monster in our game, we need to manage several key pieces of information. This data will define the monster's current status and abilities during a battle. The essential variables to track are:

- **Health and Energy:** The current health and energy points of the monster.
- **Level:** The monster's current level, which influences its stats and abilities.
- **Unlocked Attacks:** A list of attacks the monster has learned based on its level. As you may recall from our data.gd file, monsters unlock new attacks at specific level milestones.

Additionally, we need an easy way to access the base information for each monster type from our main monster_data dictionary, such as its maximum HP, element type, and icon texture.

Introduction to Godot Resources

To manage this data effectively, we will use a built-in Godot feature called a Resource. A resource is essentially a data container. More specifically, it can be thought of as a script that is not attached to a node in the scene tree. While it won't be part of the scene, it still benefits from standard GDScript features like exported variables and functions. This makes it an ideal structure for tracking our monster data, where each individual monster will be represented by one resource instance.

Creating the Monster Resource Script

Let's begin by creating the blueprint for our monster data. First, create a new folder named resources in the FileSystem dock. Then, follow these steps:

1. Right-click the new resources folder and select New -> Script.
2. Name the script monster_res.gd.
3. For the "Inherits" field, type Resource.
4. Click "Create".

Your new script file, monster_res.gd, will be very simple, containing only one line that specifies it extends from Resource.

```
extends Resource
```

This script will act as a template for all our monster resources. However, if you try to create a new resource from this script (Right-click -> New -> Resource), you will find that it doesn't appear in the list of available types. To make it accessible, we need to register it as a custom class using the class_name keyword.

Add the following line at the top of your monster_res.gd script:

```
class_name MonsterResource  
extends Resource
```



Now, if you save the file and try to create a new resource again, you can search for “MonsterResource” and it will appear as an option.

Creating a Resource Instance

With our blueprint ready, let’s create a test instance:

1. Right-click the resources folder.
2. Select New -> Resource.
3. In the dialog, search for and select MonsterResource.
4. Name the new file Test.tres and save it.

You now have a Test.tres file. If you select it, you’ll notice the Inspector is empty. This is because our resource script doesn’t have any properties yet.

Adding Data to the Resource

Let’s add some variables to our MonsterResource script. We will export them so they can be edited directly in the Inspector. We need an ID to identify the monster type and a level. Open `monster_res.gd` and add the following variables:

```
class_name MonsterResource
extends Resource

@export var id: Data.Monster
@export var level: int
```

After saving, select Test.tres again. The Inspector now displays fields for “Id” and “Level”. The “Id” field is a dropdown menu containing all the monster types from our Data.Monster enum, and “Level” is an integer field. For testing, you can select a monster and set its level to 10.

Using the Resource in a Scene

Now we can use this resource within our battle scene. We will link it to the BattleSprite node. Open the script attached to your battle sprite and export a variable of type MonsterResource.

```
@export var res: MonsterResource
```

With the battle sprite node selected in the scene, you can now drag the Test.tres file from the FileSystem dock and drop it into the “Monster Res” slot in the Inspector.

To verify that the data is accessible, we can print its values when the scene starts. In the battle sprite’s script, add a `_ready` function:

```
func _ready():
    print(res.id)
    print(res.level)
```



When you run the scene, the console will output the enum value for the selected monster (e.g., 0 for the first monster in the enum) and its level (10). This confirms that our battle sprite can successfully read the data from the resource file.

Dynamically Setting the Sprite Texture

The real power of this setup is using the resource data to configure our nodes dynamically. We can use the monster's ID to fetch the correct texture from our `monster_data` dictionary and apply it to the sprite.

Modify the `_ready` function in the battle sprite's script to load the texture:

```
func _ready():  
    # Load the texture from the path and assign it to the sprite  
    texture = load(Data.monster_data[res.id]['battle texture'])
```

Now, when you run the scene, the battle sprite will display the texture corresponding to the monster ID set in `Test.tres`. If you want to change the monster, you only need to double-click `Test.tres` and select a different monster from the "Id" dropdown in the Inspector. Running the scene again will show the updated monster without any further code changes.

This resource-based approach provides a flexible and powerful way to manage game data. Later, we will expand the `MonsterResource` script with more data and functions to handle health, energy, and attacks, making it the central hub for all monster-related logic.



In our previous lessons, we established a basic battle system. However, every monster sprite was assigned the same, hardcoded resource. This is not ideal for a dynamic game. In this lesson, we will refactor our code to allow each monster to have its own unique data, such as its type and level, by creating and passing around individual monster resources.

Storing Monster Data

First, we need a place to store the data for the player's and the enemy's teams. We will manage this in our global `data.gd` script. Let's define two new array variables to hold the monster data for each side.

In data.gd, add the following variables:

```
var player_monsters = []
var enemy_monsters = []
```

These arrays will hold instances of our MonsterResource. To easily create and configure these resources, we will also add a helper function to `data.gd`.

```
func new_monster_res(id, level):
    var monster_res = MonsterResource.new()
    monster_res.setup(id, level)
    return monster_res
```

This function, new_monster_res, takes a monster ID and a level, creates a new MonsterResource, calls a setup function on it (which we will create shortly), and returns the configured resource.

Now, we can populate our arrays with some sample data. For now, let's give the player and the enemy three monsters each with varying types and levels.

```
var player_monsters = [
    new_monster_res(Monster.SPARCHU, 20),
    new_monster_res(Monster.ATROX, 21),
    new_monster_res(Monster.JACANA, 22),
]
var enemy_monsters = [
    new_monster_res(Monster.CINDRILL, 20),
    new_monster_res(Monster.VULKEO, 21),
    new_monster_res(Monster.GULFIN, 22),
]
```

Configuring the Monster Resource

Our new_monster_res function calls a setup method that doesn't exist yet. Let's add it to our monster_res.gd script. This function will be responsible for initializing the resource's properties.

Open monster_res.gd and add the setup function:

```
func setup(new_id, new_level):
```

```
id = new_id
level = new_level
```

Passing Data to Battle Sprites

With our data structures in place, we need to pass this information to the battle scene to create the correct sprites. We will start in the battle.gd script.

In the `_ready` function, modify the calls to `battle_sprite_setup` to include our new data arrays as an argument.

```
func _ready() -> void:
    $BG.texture = load(Data.biome_bg[Data.current_biome])
    battle_sprite_setup($StartPositions/Player, $BattleSprites/Player, true, Data.player_monsters) # player
    battle_sprite_setup($StartPositions/Enemy, $BattleSprites/Enemy, false, Data.enemy_monsters) # enemy
```

Next, we must update the `battle_sprite_setup` function to handle this new data parameter. We also need to adjust its logic. Previously, it assumed exactly three monsters. Now, it must be flexible enough to handle any number of monsters, up to the number of available starting positions.

Here is the updated `battle_sprite_setup` function in battle.gd:

```
func battle_sprite_setup(start_positions: Node2D, parent: Node2D, is_player: bool, data: Array):
    for start_pos_index: int in start_positions.get_child_count():
        if start_pos_index < data.size():
            var pos = start_positions.get_child(start_pos_index).position
            var monster_res: MonsterResource = data[start_pos_index]
            create_battle_sprite(pos, parent, is_player, monster_res)
```

Notice a few key changes:

- The loop now iterates from 0 up to the number of child nodes in `start_positions`.
- An if statement checks if a corresponding monster exists in the data array for the current index. This prevents errors if you have fewer monsters than start positions.
- We retrieve the specific `MonsterResource` from the data array.
- This resource is then passed to the `create_battle_sprite` function.

We now need to update `create_battle_sprite` to accept the resource and pass it along to the battle sprite instance.

```
func create_battle_sprite(pos: Vector2, parent: Node2D, is_player: bool, res: MonsterResource):
    var battle_sprite = battle_sprite_scene.instantiate()
    battle_sprite.setup(pos, is_player, res)
    parent.add_child(battle_sprite)
```



Setting Up the Battle Sprite

The final step is to modify the BattleSprite scene's script, `battle_sprite.gd`, to use the resource it receives.

1. First, declare a variable at the top of the script to hold the monster resource.
2. Next, update the setup function to accept the resource and store it.
3. Finally, use the data from the stored resource to load the correct texture.

Here are the changes for `battle_sprite.gd`:

```
extends Sprite2D
var monster_res: MonsterResource
# ... other variables

func setup(pos: Vector2, is_player: bool, res: MonsterResource):
    position = pos
    flip_h = is_player
    monster_res = res
    texture = load(Data.monster_data[res.id]['battle texture'])
```

We have removed the old hardcoded resource loading and replaced it with a dynamic lookup. The script now accesses the monster's ID from the passed-in resource (`res.id`) and uses it to find the correct texture path in our global `monster_data` dictionary.

If you run the scene now, you should see different monsters for the player and the enemy, corresponding to the data you defined in `data.gd`. You can easily change the teams by modifying the `player_monsters` and `enemy_monsters` arrays, demonstrating the new flexibility of our system.



With the basic scene established, we can now begin working on the battle system. To make the game interactive, we will need to implement several UI elements. Each monster will have its own UI to display its name, level, health, energy, and readiness. Additionally, a battle menu is required for player input. As these are extensive features, we will first address a few smaller, preparatory tasks in this lesson.

Customizing the Battle Background

Currently, our battle scene always uses the same grass background. To make the environment more dynamic, we will implement a system to change the background based on the current biome. We can achieve this by mapping our existing Biome enum to different background images.

First, open the data.gd script. We will add a new constant dictionary to associate each biome with a specific background texture path.

```
const biome_bg = {  
  Biome.GRASS: "res://graphics/backgrounds/background1.png",  
  Biome.DESERT: "res://graphics/backgrounds/background2.png",  
  Biome.ICE: "res://graphics/backgrounds/background3.png",  
}
```

Next, we need a variable to store the current biome for the battle. Still in data.gd, add the following variable. We will set its default value to Biome.GRASS for now.

```
var current_biome: Biome = Biome.GRASS
```

Now, we can use this data to dynamically set the background in our battle scene. Open the battle.gd script. In the `_ready` function, before setting up the monster sprites, add the following line to update the background texture.

```
func _ready() -> void:  
  $BG.texture = load(Data.biome_bg[Data.current_biome])  
  # ... rest of the function
```

The `load()` function is necessary here because our dictionary stores the file path as a string. `load()` takes this path and returns the actual texture resource, which can then be assigned to the sprite's texture property. If you run the game, the background should be the grass field. If you change the `current_biome` in data.gd to `Biome.DESERT` and run it again, you will see the desert background, confirming our system works.

Adjusting the Viewport

The current game window is a bit small. To provide more space for our UI elements, we can increase its size.

1. Go to **Project > Project Settings**.
2. Under the **Display > Window** section, set the **Viewport Width** to 1920 and **Viewport Height** to 1080.
3. To compensate for the larger window size, select the **Camera2D** node in the Battle scene.



4. In the Inspector, change the **Zoom** property for both X and Y to 3. This should make the camera's outline align with the edges of the background sprite.

Setting Up the Monster UI

Each monster in the battle needs its own UI. We will build this UI directly within the battle_sprite.tscn scene.

1. Open battle_sprite.tscn.
2. Add a **Control** node as a child of the root BattleSprite node.
3. Since a Control node behaves differently from a Node2D, we need to define a fixed area for it. With the Control node selected, go to the Inspector and find the **Layout > Transform** section. Set the **Position** to be centered on the sprite.
4. Under **Layout > Custom Minimum Size**, set the size to 80 for X and 120 for Y. This creates a fixed-size UI canvas for each monster that will not scale with the window resolution.

Displaying the Monster's Name

The first UI element we will add is the monster's name.

- Add a **PanelContainer** as a child of the Control node. Rename it to NameContainer.
- In the Layout menu at the top of the 2D viewport, anchor it to the **Center Top** position.
- In the Inspector, under **Layout > Anchor Offsets**, set all four values (Left, Top, Right, Bottom) to 0.
- Add a **MarginContainer** as a child of the NameContainer.
- Finally, add a **Label** as a child of the MarginContainer.
- To give the name some padding, select the MarginContainer and under **Theme Overrides > Constants**, set the margins for Left, Top, Right, and Bottom to 4, 2, 4, and 2 respectively.

With the nodes in place, let's set the name dynamically from the battle_sprite.gd script. We will create a new function called ui_setup to handle all UI-related initialization.

```
func ui_setup():
    # name
    $Control/NameContainer/MarginContainer/Label.text = Data.monster_data[monster_res.id][ 'name' ]
```

Remember to call this new function from your existing setup function.

```
func setup(pos: Vector2, is_player: bool, res: MonsterResource, new_index: int):
    position = pos
    flip_h = is_player
    monster_res = res
    texture = load(Data.monster_data[res.id][ 'battle texture' ])
    ui_setup()
```

Creating a Readiness Bar

Next, we will add a progress bar to indicate when a monster is ready to perform an action.

- Add a **TextureProgressBar** as a child of the Control node. Rename it to ReadyProgressBar.



- Anchor it to the **Center Left** position.
- In the Inspector, set the **Fill Mode** to **Bottom to Top**.
- Under **Textures**, assign the vbar.png file (from graphics/UI/) to both the **Under** and **Progress** properties.
- To distinguish the two layers, we will tint them. Under **Tint**, set the **Under** color to a dark gray (#1f1f1f) and the **Progress** color to yellow (#ffff6a).
- Finally, under **Layout > Anchor Offsets**, set all values to 0 to ensure it is perfectly centered on the left side.

Implementing Readiness Logic

To make the readiness bar functional, we need to increase its value over time. We can do this within the `_process` function in `battle_sprite.gd`. For now, we will use a simple implementation that increases the readiness of all monsters at a constant rate.

First, add a new variable to the script.

```
var paused: bool
```

Now, add the `_process` function to update the progress bar's value on each frame.

```
func _process(delta: float) -> void:  
    if not paused:  
        $Control/ReadyProgressBar.value += 20 * delta
```

Since `paused` is always false, this logic will apply to all monster sprites. When you run the game, you should see the yellow readiness bars for all monsters filling up. In a future lesson, this hardcoded value will be replaced with a system that uses each monster's speed stat to determine how quickly they become ready.



In this lesson, we will continue developing our battle system. Our primary focus will be on making the monster's readiness dependent on its individual speed stat. We will also implement the logic to pause the battle when a monster is ready for its turn and begin creating the user interface for player actions.

Using Monster-Specific Stats

Currently, all monsters become ready at the same rate. To make the battle more dynamic, we need to use each monster's unique speed stat. The best place to manage stat retrieval is within the monster resource itself.

Let's open the `monster_res.gd` script and add a new function called `get_stat`. This function will take a string representing the desired stat (e.g., "speed", "max hp") and return its value, scaled by the monster's level.

```
func get_stat(stat: String):  
    return Data.monster_data[id]['stats'][stat] * level
```

This function accesses the global `Data.monster_data` dictionary, finds the correct monster using its id, retrieves the specific stat from the 'stats' dictionary, and multiplies it by the monster's level before returning the result.

Updating the Battle Sprite Readiness

Now, let's use this new function in our `battle_sprite.gd` script. We need to modify the `_process` function to update the readiness bar based on the monster's speed. However, directly setting the progress bar's value with the small, delta-adjusted speed value can lead to precision issues where the bar doesn't appear to move for monsters with low speed.

To solve this, we will introduce a new class variable, `readiness`, to accumulate the progress as a float. This accumulated value will then be assigned to the progress bar.

First, declare the readiness variable at the top of your `battle_sprite.gd` script.

```
extends Sprite2D  
var monster_res: MonsterResource  
var paused: bool  
var readiness: float  
# ... rest of the variables
```

1. Next, update the `_process` function to use this new variable.

```
func _process(delta: float) -> void:  
    if not paused:  
        readiness += monster_res.get_stat('speed') * delta  
        $Control/ReadyProgressBar.value = readiness
```

With this change, each monster's readiness bar will now fill up at a rate determined by its own speed stat, providing a more accurate and visually consistent progression.

Pausing the Battle on Monster Readiness

Once a monster's readiness bar is full (reaches a value of 100), it's their turn to act. At this point, we need to pause all other monsters to prevent them from continuing to build readiness.

An effective way to manage all battle sprites is to add them to a group.

1. Select the root node of your `battle_sprite.tscn` scene.
2. Go to the "Node" tab in the inspector dock, select "Groups", and add a new group named "BattleSprites".
3. Next, we need to react when the readiness bar's value changes. Select the `ReadyProgressBar` node within the `battle_sprite` scene.
4. In the "Node" tab, go to "Signals" and connect the `value_changed` signal to the `battle_sprite.gd` script. Godot will create a new function, `_on_ready_progress_bar_value_changed`.
5. Implement the logic inside this new function. We will check if the value has reached 100. If it has, we will iterate through every node in the "BattleSprites" group and set its `paused` property to `true`. We will also reset the current monster's readiness bar to 0.

```
func _on_ready_progress_bar_value_changed(value: float) -> void:
    if value >= 100:
        for battle_sprite in get_tree().get_nodes_in_group("BattleSprites"):
            battle_sprite.paused = true
            $Control/ReadyProgressBar.value = 0
```

To make testing easier, you can temporarily increase the level of your player's monsters in `data.gd` to ensure they are faster than the enemy monsters.

Creating the Battle Menu

When a player's monster is ready, a menu should appear, allowing the player to choose an action. Let's start building this interface.

1. Create a new scene. Choose "User Interface" as the root node (which creates a `Control` node). Name it `BattleMenu` and save it as `battle_menu.tscn` inside the `scenes/battle` folder.
2. To ensure the menu appears centered and does not scale with the window size, select the `BattleMenu` node.
3. Add another `Control` node as a child of `BattleMenu` and name it `MainSelection`. This will hold our main action buttons. Apply the same "Center" anchor preset to this node as well. We also want to set the `Anchor Offsets` to all be 0px.

Creating a Reusable Button

Our menu will have four main buttons: Attack, Defend, Swap, and Catch. We can create a reusable scene for these buttons.

1. Create another new scene with a `TextureButton` as the root. Name it `BattleMainButton` and save it in `scenes/battle`.
2. Like the menu, set its "Anchors Preset" to "Center" to prevent scaling issues and the `Anchor Offsets` to 0px.
3. In the Inspector, find the "Textures" property. We will assign textures for different button states. You can find the required assets in `graphics/UI/`. For this example, we'll set up the "Attack" button:
 - **Normal:** Assign the standard attack button texture (e.g., `main_button_attack.png`).
 - **Pressed:** Assign the highlighted version of the texture.



- **Hover:** Assign the highlighted version as well.
- **Focused:** Also assign the highlighted version. This ensures the button appears highlighted when selected via keyboard or controller.

Assembling the Main Menu

Now, let's return to the BattleMenu scene and add our buttons.

1. Instance the BattleMainButton scene four times as a child of the MainSelection node.
2. Rename each instance to reflect its function: Attack, Defend, Swap, and Catch.
3. For each button, update its "Textures" properties to use the correct graphics (shield for Defend, arrows for Swap, hand for Catch). Remember to set the highlighted texture for the Pressed, Hover, and Focused states on each button.

Finally, instance your new BattleMenu scene into the main Battle scene. For now, it will appear in the center of the screen when the battle starts, but it won't have any functionality yet. We will bring it to life in the upcoming lessons.



In this lesson, we will implement the functionality for our battle menu to appear when a player-controlled monster is ready to take its turn. This involves creating a communication system between the monster's battle sprite and the main battle scene, which will then trigger the menu to display at the correct position.

Signaling Monster Readiness

Currently, our BattleSprite nodes track their readiness with a progress bar. When this bar reaches 100, the monster is ready for action. We need a way for the BattleSprite to notify the main Battle scene of this event. In Godot, signals are the ideal tool for this kind of communication.

First, let's open the battle_sprite.gd script. We will define a new signal at the top of the script. This signal will pass the BattleSprite instance itself as an argument, so the receiver knows which monster is ready. We also want to add a new playable variable to keep track of whether or not the player can control that battle sprite.

```
extends Sprite2D
var monster_res: MonsterResource
var paused: bool
var readiness: float
var playable : bool

signal is_ready(battle_sprite: Sprite2D)
```

Next, we need to emit this signal when the readiness progress bar is full. Locate the _on_ready_progress_bar_value_changed function and add the line to emit the signal.

```
func _on_ready_progress_bar_value_changed(value: float) -> void:
    if value >= 100:
        for battle_sprite in get_tree().get_nodes_in_group("BattleSprites"):
            battle_sprite.paused = true
            $Control/ReadyProgressBar.value = 0
            is_ready.emit(self)
```

Handling the Signal in the Battle Scene

Now that the BattleSprite emits a signal, the main Battle scene needs to listen for it. We will connect to this signal when we first create the battle sprite.

In battle.gd, find the create_battle_sprite function and add a line to connect the new is_ready signal to a new handler function, which we will name monster_ready.

```
func create_battle_sprite(pos: Vector2, parent: Node2D, is_player: bool, res: MonsterResource, index: int):
    var battle_sprite = battle_sprite_scene.instantiate()
    battle_sprite.setup(pos, is_player, res)
    parent.add_child(battle_sprite)
    battle_sprite.connect("is_ready", monster_ready)
```

Next, we need to create the `monster_ready` function itself. This function will receive the ``battle_sprite`` that emitted the signal. We'll also add a new variable, `current_battle_sprite`, to keep track of which monster's turn it is.

```
extends Node2D
```

```
var battle_sprite_scene = preload("res://scenes/battle/battle_sprite.tscn")
var current_battle_sprite: Sprite2D
# ...
```

```
func monster_ready(battle_sprite: Sprite2D):
    current_battle_sprite = battle_sprite
    if battle_sprite.playable:
        $BattleMenu.reveal(battle_sprite)
```

This code stores the active monster and then checks if it is a playable monster. If it is, we call a new function named `reveal` on our `BattleMenu` node, passing the active monster's sprite. This `reveal` function does not exist yet, so let's create it.

Creating the Battle Menu Script

Select the `BattleMenu` node in the Scene dock and attach a new script to it called `battle_menu.gd`. Inside this new script, we will define the `reveal` function.

For now, we will just print the received `battle_sprite` to confirm that our signal and connection are working correctly.

```
extends Control
```

```
func reveal(battle_sprite: Sprite2D):
    print(battle_sprite)
```

If you run the game now, you will encounter an error: "Invalid get index 'playable' (on base: 'Sprite2D')". This is because we are trying to access the `playable` property on our `BattleSprite`, but we have not yet assigned it a value.

Adding the `playable` Property

To fix the error, we need to properly initialize the `playable` variable in `battle_sprite.gd`. We can do this inside the `setup` function, assigning it the value of the `is_player` parameter.

```
func setup(pos: Vector2, is_player: bool, res: MonsterResource, new_index: int):
    position = pos
    flip_h = is_player
    playable = is_player
    monster_res = res
    texture = load(Data.monster_data[res.id]['battle texture'])
    ui_setup()
```



With this change, running the game again should show the Sprite2D object printed to the output console when a player monster's readiness bar fills up, confirming our setup is working.

Positioning and Populating the Menu

Now we can implement the actual logic for the reveal function in `battle_menu.gd`. The menu should appear at the location of the active monster. We also need to get the monster's data for when we add buttons for attacks and other actions.

Update the `battle_menu.gd` script as follows:

1. Create a variable to hold the monster's resource data.
2. In the reveal function, set the menu's position to the battle sprite's position.
3. Assign the battle sprite's `monster_res` to our new variable.

```
extends Control
```

```
var monster_res: MonsterResource
```

```
func reveal(battle_sprite: Sprite2D):  
    position = battle_sprite.position  
    monster_res = battle_sprite.monster_res
```

Controlling Menu Visibility

Finally, the battle menu should not be visible at the start of the battle. It should only appear when its reveal function is called. To achieve this, we can hide the entire `BattleMenu` node by default and then show it inside reveal.

A simple way to do this is to hide the node in the Scene view and call `show()` in the reveal function.

```
extends Control
```

```
var monster_res: MonsterResource
```

```
func reveal(battle_sprite: Sprite2D):  
    position = battle_sprite.position  
    monster_res = battle_sprite.monster_res  
    show()
```

Now, when you run the game, the battle menu will remain hidden until a player's monster is ready. At that point, it will appear centered on that monster, ready for the next steps of development.



In this lesson, we will bring our battle menu buttons to life. Currently, they are all positioned at the center of the battle sprite. Our goal is to arrange them in a visually appealing semi-circle, enable focus on the first button, and dynamically adjust the available options based on the type of battle.

Managing Menu States

Our battle menu will have several states, such as the main selection, the attack selection, and the monster swap menu. To manage these, we will implement a simple state machine. First, let's define a state variable in the battle_menu.gd script. This variable will track the current menu state using our predefined Data.MenuState enum.

We will also use a setter function, which is a special function that runs automatically whenever the variable's value is changed. This is a powerful Godot feature that helps keep our code organized.

1. Open battle_menu.gd.
2. Declare a new variable named state and assign it a setter function called state_handler.

```
var state: Data.MenuState: set = state_handler
```

Next, whenever the menu is displayed via the reveal function, we must set its initial state to MAIN.

```
func reveal(battle_sprite: Sprite2D, reserve_monsters: Array):
    show()
    state = Data.MenuState.MAIN
    position = battle_sprite.position
    monster_res = battle_sprite.monster_res
    monster_list = reserve_monsters
```

Implementing the State Handler

The state_handler function will be the core of our menu's logic. It will receive the new state as a parameter, hide all menu elements, and then selectively show the ones corresponding to the new state.

1. Create the state_handler function. It must accept one parameter for the new state.
2. The first line in the function should assign the new state value to the state variable.
3. Next, loop through all direct children of the BattleMenu node and hide them.
4. Use a match statement to handle the logic for each state. For now, we will only implement the MAIN state, which will show the MainSelection node and call a new function, main_button_setup, to position the buttons.

```
func state_handler(new_state):
    state = new_state
    for node in get_children():
        node.hide()
    match state:
        Data.MenuState.MAIN:
            $MainSelection.show()
            main_button_setup()
```

Positioning the Main Menu Buttons

The `main_button_setup` function will arrange our buttons in a semi-circle. We will achieve this by starting with a horizontal vector and rotating it incrementally for each button.

First, let's create a constant to hold the angle offset between buttons. This avoids using "magic numbers" and makes the code easier to adjust.

```
const offset = 24
```

Now, let's create the `main_button_setup` function. We will loop through each button, calculate its rotation, and set its position. The formula $(\text{offset} * i) - \text{offset} * 1.5$ helps center the arc of buttons. We also add a small vertical offset to position the entire arc correctly relative to the monster sprite.

```
func main_button_setup():
    for i: int in $MainSelection.get_child_count():
        var button = $MainSelection.get_child(i) as TextureButton
        var angle = deg_to_rad((offset * i) - offset * 1.5)
        button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)
```

Handling Trainer Fights

The "Catch" option should not be available when fighting a trainer. To handle this, we first need a global flag to indicate the battle type. Open `data.gd` and add the following variable.

```
var trainer_fight: bool = false
```

Back in `battle_menu.gd`, we will modify `main_button_setup` to check this flag. If it's a trainer fight, we will only position the first three buttons and hide the fourth ("Catch") button. The angle calculation is also adjusted slightly to create a balanced arc with three buttons.

```
func main_button_setup():
    if Data.trainer_fight:
        for i in 3:
            var button = $MainSelection.get_child(i) as TextureButton
            var angle = deg_to_rad((offset * i) - offset)
            button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)
            $MainSelection.get_child(-1).hide()
    else:
        for i: int in $MainSelection.get_child_count():
            var button = $MainSelection.get_child(i) as TextureButton
            var angle = deg_to_rad((offset * i) - offset * 1.5)
            button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)
```

Focusing the First Button

To allow for immediate keyboard or controller navigation, we need to automatically focus the first button when the menu appears. We can do this with the `grab_focus()` method.



Sometimes, Godot may need a moment to process node updates before focus can be grabbed reliably. To ensure this works every time, we will wait for the next process frame before calling `grab_focus()`. This code should be placed after the if/else block so it runs in both trainer and wild battles.

Here is the complete `main_button_setup` function:

```
func main_button_setup():
    if Data.trainer_fight:
        for i in 3:
            var button = $MainSelection.get_child(i) as TextureButton
            var angle = deg_to_rad((offset * i) - offset)
            button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)
        $MainSelection.get_child(-1).hide()
    else:
        for i: int in $MainSelection.get_child_count():
            var button = $MainSelection.get_child(i) as TextureButton
            var angle = deg_to_rad((offset * i) - offset * 1.5)
            button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)

    await get_tree().process_frame
    $MainSelection.get_child(0).grab_focus()
```

Run the game now. You will see the buttons correctly positioned in an arc. The first button will be highlighted, and you can use the arrow keys to navigate between them. If you set `trainer_fight` to true in `data.gd` and run the game again, you will see only three buttons available.



Now that the basic buttons for our battle menu are in place, it is time to make them functional. To achieve this, the buttons must emit a signal when pressed, carrying information about what action they represent. This signal will then be handled by the main battle menu script to trigger the appropriate game logic.

Scripting the Main Buttons

First, we will attach a script to our `battle_main_button` scene. This script will manage the button's state and its intended action. It will also define a custom signal to communicate with the battle menu.

In the new script, `battle_main_button.gd`, let's add a few variables and a signal. The state variable will hold the current menu state (e.g., `MAIN`), while the info variable will store the target state the button should transition to (e.g., `ATTACK`). We will also define a custom signal named `press` that will be emitted when the button is clicked.

```
extends TextureButton

var state: Data.MenuState
var info: Data.MenuState
signal press(state: Data.MenuState, info: Data.MenuState)

func _on_pressed() -> void:
    press.emit(state, info)
```

We connect the button's built-in `pressed` signal to the `_on_pressed` function. Inside this function, we emit our custom `press` signal, passing along the button's current state and info.

Connecting Buttons in the Battle Menu

Next, we need to listen for this new `press` signal in our `battle_menu.gd` script. We will create a new function to handle the button presses and connect each button's signal to it. To avoid connecting the signal multiple times (since `main_button_setup` might be called more than once), we will add a check to see if the signal is already connected.

In `battle_menu.gd`, update the `main_button_setup` function and add a new `button_handler` function.

```
func main_button_setup():
    # ... existing code for positioning buttons ...

    await get_tree().process_frame
    $MainSelection.get_child(0).grab_focus()
    for button in $MainSelection.get_children():
        if not button.is_connected('press', button_handler):
            button.connect('press', button_handler)

func button_handler(current_state: Data.MenuState, info):
    if current_state == Data.MenuState.MAIN and info == Data.MenuState.DEFEND:
        defend.emit()
```



The `button_handler` function receives the `current_state` and info from the button that was pressed. For now, we will focus on the “Defend” action. Notice that the `info` parameter is not strictly typed. This is intentional, as this handler will later receive different types of information, such as specific attacks, not just menu states.

Assigning Information to Buttons

Currently, the `state` and `info` variables on our buttons are not assigned any values. Let’s fix this by defining the relationships in `battle_menu.gd` and assigning them within the `main_button_setup` function. We will use a dictionary to link a button’s index to a specific menu state.

At the top of `battle_menu.gd`, add the `button_link` dictionary:

```
const button_link = {
    0: Data.MenuState.ATTACK,
    1: Data.MenuState.DEFEND,
    2: Data.MenuState.SWAP,
    3: Data.MenuState.CATCH,
}
```

Now, modify the loops inside `main_button_setup` to assign the correct state and info to each button.

```
func main_button_setup():
    if Data.trainer_fight:
        for i in 3:
            var button = $MainSelection.get_child(i) as TextureButton
            var angle = deg_to_rad((offset * i) - offset)
            button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)
            button.state = Data.MenuState.MAIN
            button.info = button_link[i]
            $MainSelection.get_child(-1).hide()
    else:
        for i: int in $MainSelection.get_child_count():
            var button = $MainSelection.get_child(i) as TextureButton
            var angle = deg_to_rad((offset * i) - offset * 1.5)
            button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)
            button.state = Data.MenuState.MAIN
            button.info = button_link[i]

# ... rest of the function
```

As a final touch for the menu, let’s hide it by default. In the `_ready` function of `battle_menu.gd`, hide the node.

```
func _ready() -> void:
    $MainSelection.hide()
```

Implementing the Defend Action



The “Defend” action is the simplest to implement, as it essentially skips the player’s turn. When the defend button is pressed, our button_handler will emit a defend signal. Let’s declare this signal in battle_menu.gd.

```
signal defend
```

Now, we need to catch this signal in our main battle.gd script. Connect the defend signal from the BattleMenu node to a new function in the Battle scene.

```
func _on_battle_menu_defend() -> void:
    $BattleMenu.finish()
    battle_continue()
```

This requires two new functions. First, the finish function in battle_menu.gd, which will hide the menu.

```
func finish():
    for node in get_children():
        node.hide()
    hide()
```

Second, the battle_continue function in battle.gd. This function will resume the battle by unpausing all monsters and clearing the current acting monster. We will also add a short delay to improve the pacing of the battle.

```
func battle_continue():
    await get_tree().create_timer(0.5).timeout
    current_battle_sprite = null
    for battle_sprite in get_tree().get_nodes_in_group('BattleSprites'):
        battle_sprite.paused = false
```

Fixing the Turn Loop

A crucial step is to ensure that once a monster takes its turn, its readiness is reset. Otherwise, it would immediately get another turn. In battle_sprite.gd, within the _on_ready_progress_bar_value_changed function, we must reset the readiness variable itself to zero.

```
func _on_ready_progress_bar_value_changed(value: float) -> void:
    if value >= 100:
        for battle_sprite in get_tree().get_nodes_in_group("BattleSprites"):
            battle_sprite.paused = true
            readiness = 0
            is_ready.emit(self)
```

Handling Enemy Turns



Our battle flow should also account for enemy turns. In `battle.gd`, we can modify the `monster_ready` function. If the monster that is ready is not a playable character, we will simulate an enemy turn. For now, this will just be a placeholder that prints a message and continues the battle.

```
func monster_ready(battle_sprite: Sprite2D):
    current_battle_sprite = battle_sprite
    if battle_sprite.playable:
        $BattleMenu.reveal(battle_sprite)
    else:
        # This block will be expanded later to handle enemy AI and attacks
        print("monster attack")
        battle_continue()
```

With these changes, we have established a basic turn-based battle flow. The player can now select the “Defend” option to skip their turn, and the system will correctly cycle through both player and enemy turns, paving the way for more complex actions like attacking and swapping monsters.



In this lesson, we will focus on creating the attack menu for our battle system. When a player chooses to attack, a new set of buttons corresponding to the monster's available attacks should appear. We will create a new scene for these buttons, set up a container to hold them, and write the logic to dynamically generate them based on the current monster's level and abilities.

Creating the Attack Button Scene

First, we need a dedicated scene for our attack buttons. This allows us to easily instantiate them as needed.

1. Create a new scene with a Button node as its root.
2. Rename the root node to BattleAttackButton.
3. Save the scene as BattleAttackButton.tscn inside the scenes/battle/ folder.
4. For now, you can set the button's text to "Attack" as a placeholder.
5. In the Inspector, under the Control section, find Size Flags. Enable the Expand option for both Horizontal and Vertical. This ensures the button will fill the space allocated to it by its parent container.

Setting Up the Attack Container

Next, we will prepare the BattleMenu scene to display these new buttons. We will use a GridContainer to arrange them neatly.

1. Open the BattleMenu scene.
2. Add a GridContainer node as a child of the BattleMenu root node.
3. Rename this new node to AttackContainer.
4. In the Inspector for the AttackContainer, set the Columns property to 2.
5. To see how it looks, you can temporarily drag four instances of your BattleAttackButton.tscn scene into the AttackContainer.

Now, let's position the container. We want it to appear on the left side of the screen.

1. Select the AttackContainer.
2. In the toolbar, go to Layout -> Anchors Preset and choose Center Left.
3. To fine-tune its position, change the Anchor Mode to Custom and set the Left offset under Anchors to 40.

Managing Menu Visibility

Currently, all menu elements are visible when the battle starts, which is not ideal. We should hide them and only show them when needed. In the battle_menu.gd script, we can hide all children of the BattleMenu node at the beginning.

```
# battle_menu.gd

func _ready() -> void:
    for node in get_children():
        node.hide()
```

This code iterates through all direct children of the BattleMenu node (which includes our MainSelection and AttackContainer) and hides them when the scene is ready.

Handling Menu State Transitions



To show the attack menu when the player clicks the “Attack” button, we need to update our menu’s state management logic. We will modify the `state_handler` and `button_handler` functions in `battle_menu.gd`.

First, let’s update the `button_handler` to be more flexible using a match statement. This will allow it to handle button presses from different menu states.

```
# battle_menu.gd

func button_handler(current_state: Data.MenuState, info):
    match current_state:
        Data.MenuState.MAIN:
            state = info

    if current_state == Data.MenuState.MAIN and info == Data.MenuState.DEFEND:
        defend.emit()
```

When a button from the main menu is pressed (`current_state` is `Data.MenuState.MAIN`), we now update the menu’s state variable with the `info` provided by the button. This change will automatically trigger our `state_handler` function.

Next, we’ll add the logic to the `state_handler` to show the attack menu.

```
# battle_menu.gd

func state_handler(new_state):
    state = new_state
    for node in get_children():
        node.hide()
    match state:
        Data.MenuState.MAIN:
            $MainSelection.show()
            main_button_setup()
        Data.MenuState.ATTACK:
            $AttackContainer.show()
            attack_button_setup()
```

When the state changes to `Data.MenuState.ATTACK`, we show the `AttackContainer` and call a new function, `attack_button_setup`, which we will create next.

Fetching Available Attacks

Before we can create the buttons, we need a way to determine which attacks a monster can use based on its level. We will add a new function to our `MonsterResource` class for this purpose.

Open the `monster_res.gd` script and add the following function:

```
# monster_res.gd

func get_attacks() -> Array:
    var attack: Array
```



```
for level_req in Data.monster_data[id]['attacks'].keys():
    if level >= level_req:
        attack.append(Data.monster_data[id]['attacks'][level_req])
return attack
```

This function iterates through the attacks listed for the monster in our global Data script. It checks if the monster's current level meets the `level\_req` (level requirement) for each attack. If it does, the attack's enum is added to an array, which is then returned.

Dynamically Generating Attack Buttons

Now we can implement the `attack_button_setup` function in `battle_menu.gd`. This function will clear any old buttons, get the current monster's attacks, and create a new button for each one.

First, let's preload the button scene at the top of the script.

```
# battle_menu.gd

extends Control
var battle_attack_button_scene = preload("res://scenes/battle/battle_attack_button.tscn")
# ...
```

Now, add the `attack_button_setup` function.

```
# battle_menu.gd

func attack_button_setup():
    for button in $AttackContainer.get_children():
        button.queue_free()

    for attack_enum in monster_res.get_attacks():
        var btn = battle_attack_button_scene.instantiate()
        btn.setup(attack_enum)
        $AttackContainer.add_child(btn)
```

This function first removes all existing children from the `AttackContainer` to ensure a clean slate. Then, it calls the `get_attacks` method on the current `monster_res`. For each returned attack enum, it instantiates a new `BattleAttackButton`, calls a `setup` function on it (which we will create next), and adds it to the `AttackContainer`.

Finalizing the Button Script

The last step is to create the script for our `BattleAttackButton` and implement its `setup` function. This function will receive an attack enum and use it to set the button's text to the correct attack name.

1. Attach a new script to the root node of your `BattleAttackButton.tscn` scene. Save it as `battle_attack_button.gd`.
2. Add the following code to the script:



```
# battle_attack_button.gd

extends Button

func setup(attack_enum: Data.Attack):
    text = Data.attack_data[attack_enum]['name']
```

The setup function takes the attack_enum as an argument. It then looks up the corresponding attack data in our global Data script and sets the button's text property to the attack's name.

With these changes, running the game and clicking the "Attack" button in a battle will now dynamically display the correct set of attack buttons for the active monster. You can test this by temporarily changing a monster's level in data.gd to see how the list of available attacks changes.



In this lesson, we will bring the attack menu to life. When a player selects an attack, we need to process that choice, which involves selecting a target for the attack. This will require us to handle button signals, manage different menu states, and create a new UI element for target selection.

Activating the Attack Buttons

First, we must ensure that pressing an attack button triggers an action. We will work within the `battle_attack_button.gd` script. We need to store which attack the button represents and emit a signal when it's pressed.

We will define a variable to hold the attack data and a custom signal to communicate the press event. The button's `setup` function will store the attack type, and when pressed, it will emit our custom signal with the appropriate menu state and attack information.

Update your `battle_attack_button.gd` script with the following code:

```
extends Button

var info: Data.Attack
signal press(state: Data.MenuState, info: Data.Attack)

func setup(attack_enum: Data.Attack, current_ep: float):
    text = Data.attack_data[attack_enum]['name']
    info = attack_enum

func _on_pressed() -> void:
    press.emit(Data.MenuState.ATTACK, info)
```

Handling Button Presses in the Battle Menu

Now that our buttons emit a signal, the `BattleMenu` needs to listen for it. In `battle_menu.gd`, within the `attack_button_setup` function, we must connect each new button's press signal to our existing `button_handler` function. We also want to ensure the first button in the list is automatically focused for better user experience.

```
func attack_button_setup():
    for button in $AttackContainer.get_children():
        button.queue_free()
    for attack_enum in monster_res.get_attacks():
        var btn = battle_attack_button_scene.instantiate()
        btn.setup(attack_enum)
        btn.connect('press', button_handler)
        $AttackContainer.add_child(btn)
    await get_tree().process_frame
    $AttackContainer.get_child(0).grab_focus()
```

Next, we need to handle the incoming information in the `button_handler`. For now, choosing an attack should transition us to a new state for selecting a target. We will define a new signal called `select` that will carry the necessary information to the main Battle scene.



First, declare the new signal at the top of battle_menu.gd:

```
signal select(state: Data.MenuState, info)
```

Then, update the button_handler function to handle the ATTACK state. Instead of performing the attack directly, it will change the menu's state to SELECT and emit our new select signal, passing along that the selection is for an attack and which attack was chosen.

```
func button_handler(current_state: Data.MenuState, info):
    match current_state:
        Data.MenuState.MAIN:
            state = info
        Data.MenuState.ATTACK:
            state = Data.MenuState.SELECT
            select.emit(Data.MenuState.ATTACK, info)
    if current_state == Data.MenuState.MAIN and info == Data.MenuState.DEFEND:
        defend.emit()
```

Implementing Target Selection

The main Battle scene is responsible for managing the overall flow. It needs to react to the select signal from the BattleMenu. Connect this signal to a new function in the battle.gd script.

In this new handler function, we'll check if the selection is for an attack. If so, we need to determine who the potential targets are. Offensive abilities target enemies, while non-offensive ones (like healing) target the player's own monsters. We can check the offensive flag in our attack data to decide.

```
func _on_battle_menu_select(state: int, info: Variant) -> void:
    if state == Data.MenuState.ATTACK:
        var attack_data = Data.attack_data[info]
        var target = $BattleSprites/Enemy if attack_data['offensive'] else $BattleSprites/P
        layer
```

Creating the Selection Indicator

To show the player which monster they are targeting, we need a visual indicator. Let's create a new scene for this.

1. Create a new scene with a Sprite2D node as the root. Name it BattleSelectSprite.
2. For the texture, you can use an appropriate UI graphic, such as the one from `res://graphics/UI/options5.png`.
3. Attach a new script named battle_select_sprite.gd.

This script will manage the selector's state and position. It needs to be hidden by default and have an activate function to show it and set it up for a specific set of targets. This function will store the target group, the action info (e.g., the attack type), and the current state. It will then position itself



over the first available target.

```
extends Sprite2D
```

```
var target: Node2D
var info
var state: Data.MenuState
var index: int
```

```
func _ready() -> void:
    hide()
```

```
func activate(new_target, new_info, new_state: Data.MenuState):
    show()
    target = new_target
    info = new_info
    state = new_state
    position = target.get_child(index).position
```

Integrating the Selector

Now, let's add the BattleSelectSprite to our main Battle scene. To keep things organized, you can place it under a new Node2D called EffectSprites. Once instantiated, we can call its activate function from our `_on_battle_menu_select` handler in `battle.gd`.

```
func _on_battle_menu_select(state: int, info: Variant) -> void:
    if state == Data.MenuState.ATTACK:
        var attack_data = Data.attack_data[info]
        var target = $BattleSprites/Enemy if attack_data['offensive'] else $BattleSprites/Player
        $EffectSprites/BattleSelectSprite.activate(target, info, Data.MenuState.ATTACK)
```

Running the game now, selecting an offensive attack like "Claw" should show the selector on the first enemy monster, while a non-offensive one like "Heal" should show it on the first player monster.

Adding Player Input

The final step is to allow the player to move the selector between different targets. We will implement this in the `_input` function of `battle_select_sprite.gd`. The logic is as follows: if the selector is visible, listen for "up" and "down" inputs. When an input is detected, update the target index, ensuring it wraps around using `posmod`, and then update the selector's position to match the newly selected target.

Add the `_input` function to `battle_select_sprite.gd`:

```
func _input(_event: InputEvent) -> void:
    if visible:
        if Input.is_action_just_pressed("ui_down") or Input.is_action_just_pressed("ui_up")
        :
```



```
var dir = Input.get_axis("ui_up", "ui_down")
index = posmod(index + dir, target.get_child_count())
position = target.get_child(index).position
```

With this, you can now navigate between all available targets on either the enemy or player side. Our attack selection flow is now functional, paving the way for executing the attacks themselves.



Now that our system for selecting different battle sprites is functional, the next logical step is to implement the attack sequence itself. This lesson presents a challenge to consolidate your knowledge and apply what you have learned to complete this core gameplay mechanic.

Your Challenge: Implementing the Attack

When a battle sprite is selected by the player, several actions need to occur to create a complete attack sequence. Your task is to implement the logic for the following points:

1. **Play the Attack Animation:** The selected battle sprite must play its attack animation. This involves ensuring the animation uses the correct frames and is positioned correctly on the screen.
2. **Animate the Attacking Monster:** The monster that is performing the attack should also play its own corresponding attack animation.
3. **Update the Monster Resource:** After an attack is executed, the data for the affected monster (the monster resource) must be updated accordingly.
4. **Enable Returning to the Main Menu:** The player should be able to cancel the action and return from the sprite selection screen to the main battle menu. This functionality should be triggered when the *escape* button is pressed.

This challenge is an excellent opportunity to become more familiar with the project's structure and logic. We encourage you to attempt to implement these four points on your own. Take your time, review previous concepts if necessary, and see how far you can get.



In this lesson, we will enhance the user experience of our battle system by implementing a “cancel” or “back” functionality. This will allow the player to navigate backward through the menu options, such as returning from target selection to the attack menu, or from the attack menu back to the main battle options. We will also address a common input-handling issue that can arise when managing menu states across different scripts.

Canceling Target Selection

Currently, once a player chooses to attack, they are taken to a target selection screen managed by `BattleSelectSprite`. However, there is no way to back out of this choice. Our first step is to add this capability.

We will start by editing the `battle_select_sprite.gd` script. We need to define a new signal that will be emitted when the player cancels their action. Then, within the `_input` function, we will check for the “`ui_cancel`” action (typically the Escape key) and emit this new signal.

1. Declare a new signal named `cancel`. It will pass the current menu state as an argument.
2. In the `_input` function, add a condition to check if the “`ui_cancel`” action was just pressed.
3. If it was, hide the selection sprite and emit the `cancel` signal, passing the current state.

Your `battle_select_sprite.gd` script should be updated as follows:

```
extends Sprite2D

var target: Node2D
var info
var state: Data.MenuState
var index: int

signal cancel(state: Data.MenuState)

# ... existing functions ...

func _input(_event: InputEvent) -> void:
    if visible:
        if Input.is_action_just_pressed("ui_down") or Input.is_action_just_pressed("ui_up"):
            :
                var dir = Input.get_axis("ui_up", "ui_down")
                index = posmod(index + dir, target.get_child_count())
                position = target.get_child(index).position
            if Input.is_action_just_pressed("ui_cancel"):
                hide()
                cancel.emit(state)
```

Connecting the Signal

With the signal created, we must connect it to a function that can handle the logic. This connection is made in the Battle scene.

1. Open your main Battle scene.
2. Select the `BattleSelectSprite` node.
3. Go to the “Node” tab in the inspector and connect its `cancel` signal to the root Battle node’s

script.

- Godot will create a new function in battle.gd, which we will name `_on_battle_select_sprite_cancel`.

Inside this new function in battle.gd, we will call a new function on our BattleMenu node, which we will create next. This function will be responsible for reverting the menu to its previous state.

```
func _on_battle_select_sprite_cancel(state: int) -> void:
    $BattleMenu.cancel(state)
```

Reverting the Battle Menu State

Now, let's implement the cancel function in battle_menu.gd. This function will receive the state from which the player is canceling (e.g., ATTACK) and will use it to set the menu back to the appropriate screen.

In battle_menu.gd, create the cancel function. It will take one argument, which we'll call new_state. Inside, we check if the state is ATTACK and, if so, set the menu's current state back to ATTACK. This change will automatically trigger our existing state_handler function to show the attack selection buttons again.

```
func cancel(new_state: Data.MenuState):
    if new_state == Data.MenuState.ATTACK:
        state = Data.MenuState.ATTACK
```

At this point, you can test the game. When you select "Attack" and are prompted to choose a target, pressing Escape should correctly return you to the attack list.

Returning to the Main Menu

The next challenge is that pressing Escape again from the attack menu does nothing. We need to implement logic to return to the main battle menu (Attack, Defend, Swap, Catch).

We can handle this in the _input function of battle_menu.gd. We will check for the "ui_cancel" input and also verify that the current menu state is one from which we can cancel (like ATTACK or SWAP). If these conditions are met, we simply change the state to MAIN.

```
func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("ui_cancel"):
        if state in [Data.MenuState.ATTACK, Data.MenuState.SWAP]:
            state = Data.MenuState.MAIN
```

Fixing a Double-Cancel Issue

If you test the game now, you will notice a problem. When you press Escape from the target selection screen, the game returns all the way to the main menu, skipping the attack menu. This happens because the single key press is processed by both BattleSelectSprite and BattleMenu within



the same frame.

To solve this, we can introduce a very short delay using a Timer. This will prevent the `_input` function in `BattleMenu` from reacting to the same key press that triggered the cancel from `BattleSelectSprite`.

1. In the `BattleMenu` scene, add a `Control` node and name it `Support`.
2. Add a `Timer` node as a child of `Support` and name it `StopTimer`.
3. In the `Inspector` for the `StopTimer`, set its **Wait Time** to 0.2 seconds and enable **One Shot**.

Now, we will modify our script to use this timer.

- When the cancel function in `battle_menu.gd` is called, we will start the timer.
- In the `_input` function, we will add a condition to only process the cancel input if the timer is not currently running.

Here is the updated code for `battle_menu.gd`:

```
func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("ui_cancel") and not $Support/StopTimer.time_left:
        if state in [Data.MenuState.ATTACK, Data.MenuState.SWAP]:
            state = Data.MenuState.MAIN

# ... other functions ...

func cancel(new_state: Data.MenuState):
    if new_state == Data.MenuState.ATTACK:
        state = Data.MenuState.ATTACK
        $Support/StopTimer.start()
```

With these changes, our cancel logic is complete and robust. Pressing `Escape` from the target selection screen will correctly return you to the attack menu, and pressing it again will take you back to the main menu, providing a smooth and intuitive user experience.



In this lesson, we will implement the visual aspects of an attack in our battle system. This involves handling player input to initiate an attack, creating a dedicated sprite to display attack animations, and triggering an animation on the attacking monster. We will connect signals and create new functions to orchestrate these visual effects.

Handling Attack Input

First, we need to detect when the player confirms their attack selection. This action is typically associated with the “accept” input, such as the Enter or Space key. We will modify the `BattleSelectSprite.gd` script to handle this.

We will add a new signal called `use`, which will be emitted when the player presses the accept key. This signal will carry the necessary information to the main battle scene: the index of the selected target, the attack information, and the current menu state.

In `BattleSelectSprite.gd`, add the new signal definition below the existing `cancel` signal.

```
signal use(index: int, info, state: Data.MenuState)
```

Next, within the `_input` function, add a check for the “`ui_accept`” action. If it’s pressed, we emit our new signal with the current data.

```
func _input(_event: InputEvent) -> void:
    if visible:
        # ... existing input checks for up/down/cancel
        if Input.is_action_just_pressed("ui_cancel"):
            hide()
            cancel.emit(state)
        if Input.is_action_just_pressed("ui_accept"):
            use.emit(index, info, state)
```

Connecting the Attack Logic

With the signal defined, we need to connect it to our main Battle scene script. Select the `BattleSelectSprite` node in the Scene dock, navigate to the Node dock, and connect the `use` signal to a new function in the `battle.gd` script. Godot will suggest a name like `_on_battle_select_sprite_use`.

Inside this new function, we will start building the logic for an attack. For now, we will focus on the visual part. We first check if the menu state is indeed `ATTACK`. Then, we determine if the attack is offensive or defensive to identify the correct target group (enemies or players). Finally, we get the specific target monster using the index provided by the signal.

```
func _on_battle_select_sprite_use(index: int, info: Variant, state: int) -> void:
    if state == Data.MenuState.ATTACK:
        var offensive = Data.attack_data[info]['offensive']
        var target_group = $BattleSprites/Enemy if offensive else $BattleSprites/Player
        var target = target_group.get_child(index) as Sprite2D
        # More to come here...
```

Creating the Attack Effect Sprite



To display attack animations, such as explosions or slashes, we will create a dedicated sprite. This approach keeps the effect logic separate and allows us to reuse a single node for all attack effects.

1. In the Battle scene, add a new Sprite2D node. A good place for it is under a new container node, for example, EffectSprites. Rename the new sprite to BattleAttackSprite.
2. Attach a new script to it named BattleAttackSprite.gd.
3. The attack animations are sprite sheets with 4 horizontal frames. In the Inspector for BattleAttackSprite, under Animation, set Hframes to 4.

Now, let's write the script for this sprite. It will have a single function, play, which takes the attack type and a position as arguments. This function will set the sprite's position, load the correct animation texture, and use a Tween to animate the frames and visibility.

Add the following code to BattleAttackSprite.gd:

```
extends Sprite2D
```

```
func play(attack: Data.Attack, pos: Vector2):  
    position = pos  
    texture = load(Data.attack_data[attack]['texture'])  
    var tween = create_tween()  
    tween.tween_property(self, "visible", true, 0)  
    tween.tween_property(self, "frame", hframes - 1, 0.8).from(0)  
    tween.tween_property(self, "visible", false, 0)
```

This script positions the sprite, loads the texture from our global Data, and then creates a tween. The tween makes the sprite visible, animates its frame property from 0 to 3 over 0.8 seconds, and finally makes it invisible again.

Animating the Attacker

Besides the attack effect, the monster performing the attack should also have an animation. We will create a new function in BattleSprite.gd for this purpose.

In BattleSprite.gd, add the attack_animation function. This function will play the "attack" animation from its AnimationPlayer, wait for it to finish, and then return to the "idle" animation.

```
func attack_animation():  
    $AnimationPlayer.play('attack')  
    await $AnimationPlayer.animation_finished  
    $AnimationPlayer.play('idle')
```

Putting It All Together

Now we can go back to battle.gd and complete the _on_battle_select_sprite_use function. We will call the play function on our BattleAttackSprite and the attack_animation function on the current attacker.

```
func _on_battle_select_sprite_use(index: int, info: Variant, state: int) -> void:  
    if state == Data.MenuState.ATTACK:  
        var offensive = Data.attack_data[info]['offensive']
```



```
var target_group = $BattleSprites/Enemy if offensive else $BattleSprites/Player
var target = target_group.get_child(index) as Sprite2D

$EffectSprites/BattleAttackSprite.play(info, target.position)
current_battle_sprite.attack_animation()
```

Final Adjustments

To ensure a clean start to our battle, make sure both the BattleSelectSprite and BattleAttackSprite are hidden by default. You can do this by selecting each node in the editor and toggling its visibility property (the eye icon) off.

With these changes, you can now run the game. When you select an attack and a target, you should see the attack effect play at the target's position, and the attacking monster will perform its own attack animation.



In addition to playing an attack animation, we must also update the monster's underlying data to reflect the damage taken. This lesson focuses on implementing the health system, calculating damage based on stats and elemental types, and visually representing health changes with a dynamic health bar.

Updating the Monster Resource

First, we need to distinguish between a monster's maximum HP and its current HP. The maximum HP is a base stat that scales with the monster's level, while the current HP is a variable that decreases during battle. Let's add a variable to track this in our MonsterResource script.

Open `monster_res.gd` and add a new variable for the current health.

```
class_name MonsterResource extends Resource
```

```
@export var id: Data.Monster
@export var level: int
var current_hp: float
```

This `current_hp` needs to be initialized when the monster is first created. In the `setup` function, we will set its initial value to be the monster's maximum HP.

```
func setup(new_id, new_level):
    id = new_id
    level = new_level
    current_hp = get_stat('max hp')
```

Applying Damage in Battle

Now, let's return to `battle.gd`. After an attack animation is triggered, we need to call a function on the target to reduce its health. We will create a new function called `change_health` in the `BattleSprite` class for this purpose.

In the `_on_battle_select_sprite_use` function within `battle.gd`, find the section for `Data.MenuState.ATTACK`. We will calculate the damage and determine the attack's element, then pass them to the target's `change_health` function.

```
func _on_battle_select_sprite_use(index: int, info: Variant, state: int) -> void:
    if state == Data.MenuState.ATTACK:
        var offensive = Data.attack_data[info]['offensive']
        var target_group = $BattleSprites/Enemy if offensive else $BattleSprites/Player
        var target = target_group.get_child(index) as Sprite2D
        $EffectSprites/BattleAttackSprite.play(info, target.position)
        current_battle_sprite.attack_animation()

        var damage = Data.attack_data[info]['amount'] * current_battle_sprite.monster_res.level * 0.25
        var element = Data.attack_data[info]['element']
        target.change_health(damage, element)
```



...

Note that the damage calculation is preliminary. A high-level monster can deal a very large amount of damage. For balancing, you can multiply this value by a smaller factor, for instance $\times 0.1$. Game balancing is a complex topic, but this allows us to test the mechanics without instantly defeating opponents.

Implementing the `change\_health` Function

Next, we need to define the `change_health` function in `battle_sprite.gd`. This function will take the damage amount and the attack's element as arguments.

```
func change_health(amount: float, element: Data.Element):
    monster_res.current_hp -= amount
    print(monster_res.current_hp, " / ", monster_res.get_stat("max hp"))
```

This initial implementation simply subtracts the damage from the monster's current HP and prints the result for verification.

Accounting for Elemental Modifiers

To make battles more strategic, we will incorporate elemental advantages and disadvantages. A fire attack should be more effective against a plant monster, for example. We have an `element_modifier` dictionary in `data.gd` for this. To use it, we need to know both the attacking element and the defending monster's element.

First, let's create a helper function in `monster_res.gd` to retrieve a monster's element.

```
func get_element() -> Data.Element:
    return Data.monster_data[id]['element']
```

Now, we can update the `change_health` function in `battle_sprite.gd` to calculate and apply the elemental modifier.

```
func change_health(amount: float, element: Data.Element):
    var modifier = Data.element_modifier[element][monster_res.get_element()]
    monster_res.current_hp -= amount * modifier
    print(monster_res.current_hp, " / ", monster_res.get_stat("max hp"))
```

Creating a Visual Health Bar

Seeing health values in the console is useful for debugging, but players need a visual indicator. We will add a health bar to the `BattleSprite` scene.



1. In the BattleSprite scene, add a VBoxContainer to the Control node and rename it StatsContainers.
2. Inside StatsContainers, add a TextureProgressBar and name it HealthBar.
3. In the Inspector for HealthBar, assign the textures for its appearance. Set the **Under** texture to res://graphics/UI/h_bar.png and the **Progress** texture to res://graphics/UI/hbar_inner.png.
4. To color the bar, go to the **Tint** properties. Set the **Under** color to a dark grey (e.g., #2B2B2B) and the **Progress** color to red (e.g., #FF3838).
5. To position the UI, select the StatsContainers node. In the Layout menu, set its anchor preset to **Center Bottom**. Then, set all its anchor offsets to zero and give the **Bottom** offset a negative value, like -10, to move it up slightly from the monster's base.

Connecting the Health Bar to Data

The health bar must be initialized with the monster's health data when the battle starts. We'll do this in the ui_setup function in battle_sprite.gd. It's crucial to set the bar's max_value before its current value.

```
func ui_setup():
    # name
    $Control/NameContainer/MarginContainer/Label.text = Data.monster_data[monster_res.id]
    ['name']
    # health
    $Control/StatsContainers/HealthBar.max_value = monster_res.get_stat("max hp")
    $Control/StatsContainers/HealthBar.value = monster_res.current_hp
```

Animating Health Changes

Finally, let's make the health bar animate smoothly when a monster takes damage. We can achieve this using a Tween. In battle_sprite.gd, modify the change_health function to replace the print statement with a tween animation.

```
func change_health(amount: float, element: Data.Element):
    var modifier = Data.element_modifier[element][monster_res.get_element()]
    monster_res.current_hp -= amount * modifier
    var tween = create_tween()
    tween.tween_property(
        $Control/StatsContainers/HealthBar,
        "value",
        monster_res.current_hp,
        0.5)
```

Now, when a monster is attacked, its health will decrease, and the health bar will smoothly animate to reflect the new value, providing clear visual feedback to the player.



In this lesson, we will finalize the core attack logic for our battle system. We will focus on ensuring the battle continues after an attack, implementing an energy system with costs for moves, and displaying this information to the player through a new UI element.

Continuing the Battle Flow

Currently, after a monster performs an attack, the battle sequence halts. We need to ensure the game continues, allowing for the next turn. We have already created a function for this purpose, `battle_continue()`. We just need to call it after an attack is executed.

In the `battle.gd` script, locate the `_on_battle_select_sprite_use` function. At the end of this function, add a call to `battle_continue()`.

```
func _on_battle_select_sprite_use(index: int, info: Variant, state: int):
    if state == Data.MenuState.ATTACK:
        # ... attack logic ...

    # ... other logic ...

    battle_continue()
```

You may notice another issue: after selecting an attack, the selection sprite remains on screen, allowing for multiple attacks in a single turn. To fix this, we need to hide the selection sprite after it has been used. Open the `battle_select_sprite.gd` script. In the `_input` function, after the `use` signal is emitted, call the `hide()` function.

```
func _input(_event: InputEvent):
    if visible:
        # ... input logic ...
        if Input.is_action_just_pressed("ui_accept"):
            use.emit(index, info, state)
            hide()
```

With these changes, the battle will correctly proceed to the next turn after an attack, and the selection UI will disappear as intended.

Displaying Monster Energy

Attacks will consume energy, so we need a visual indicator for the monster's current energy points (EP). We will add an energy bar next to the existing health bar.

1. In the `BattleSprite` scene, inside the `StatsContainers` node, add a new `Control` node. This will help us position the energy bar.
2. Set the `Control` node's **Custom Minimum Size** on the Y-axis to 6 to create a small gap.
3. Select the parent `StatsContainers` node. Under **Theme Overrides > Constants**, set the **Separation** to -4 to bring the health and energy bars closer together.
4. Add a `TextureProgressBar` as a child of the new `Control` node. Rename it to `EnergyBar`. Ensure it fills the entire space of its parent.
5. Configure the `EnergyBar` textures. Use `Hbar.png` for the **Under** texture and `Hbar_inner.png` for the **Progress** texture.
6. To color the bar, go to the **Tint** properties. For **Progress**, use the color code `#38cdff`.



7. To give the UI a more dynamic look, select the EnergyBar and change its **Layout > Transform > Anchors Preset** to **Custom**. Then, set the **Left** anchor offset to 4 pixels.

Implementing Energy Mechanics

With the UI in place, we need to implement the underlying logic. First, we must track each monster's current energy. In the `monster_res.gd` script, add a variable for current energy points and initialize it in the `setup` function.

```
class_name MonsterResource extends Resource
```

```
# ... other variables ...  
var current_ep: float
```

```
func setup(new_id, new_level):  
    id = new_id  
    level = new_level  
    current_hp = get_stat('max hp')  
    current_ep = get_stat("max ep")
```

Next, we'll connect this data to the UI. In `battle_sprite.gd`, update the `ui_setup` function to initialize the EnergyBar with the monster's maximum and current EP.

```
func ui_setup():  
    # name  
    $Control/NameContainer/MarginContainer/Label.text = Data.monster_data[monster_res.id][  
'name']  
    # health  
    $Control/StatsContainers/HealthBar.max_value = monster_res.get_stat("max hp")  
    $Control/StatsContainers/HealthBar.value = monster_res.current_hp  
    # energy  
    $Control/StatsContainers/Control/EnergyBar.max_value = monster_res.get_stat('max ep')  
)  
    $Control/StatsContainers/Control/EnergyBar.value = monster_res.current_ep
```

Managing Energy Changes

We need a function to modify a monster's energy. In `battle_sprite.gd`, create a new function called `change_energy`. This function will handle both spending and gaining energy. It will also clamp the value to ensure it never goes below zero or above the maximum.

```
func change_energy(amount: float):  
    monster_res.current_ep += amount  
    monster_res.current_ep = clamp(monster_res.current_ep, 0, monster_res.get_stat("max ep"))  
    $Control/StatsContainers/Control/EnergyBar.value = monster_res.current_ep
```

By adding the amount, we can pass a negative value to decrease energy (when attacking) and a positive value to increase it (when recovering).



Applying Attack Costs

Now, let's make attacks consume energy. Back in `battle.gd`, inside the `_on_battle_select_sprite_use` function, we will retrieve the attack's cost and apply it to the attacking monster.

```
func _on_battle_select_sprite_use(index: int, info: Variant, state: int):
    if state == Data.MenuState.ATTACK:
        var offensive = Data.attack_data[info]['offensive']
        var target_group = $BattleSprites/Enemy if offensive else $BattleSprites/Player
        var target = target_group.get_child(index) as Sprite2D
        $EffectSprites/BattleAttackSprite.play(info, target.position)
        current_battle_sprite.attack_animation()

        var damage = Data.attack_data[info]['amount'] * current_battle_sprite.monster_res.level * 0.255
        var element = Data.attack_data[info]['element']
        var cost = Data.attack_data[info]['cost']
        target.change_health(damage, element)
        current_battle_sprite.change_energy(-cost)
    # ...
    battle_continue()
```

Notice that we pass `-cost` to the `change_energy` function, as the cost defined in our data is a positive number representing the amount of energy to subtract.

Animating the Energy Bar

Finally, to make the energy change smoother, we can animate the energy bar using a tween, just like we did for the health bar. Update the `change_energy` function in `battle_sprite.gd` to use a tween.

```
func change_energy(amount: float):
    monster_res.current_ep += amount
    monster_res.current_ep = clamp(monster_res.current_ep, 0, monster_res.get_stat("max ep"))
    var tween = create_tween()
    tween.tween_property(
        $Control/StatsContainers/Control/EnergyBar,
        "value",
        monster_res.current_ep,
        0.5)
```

Now, when a monster attacks, its energy bar will decrease gradually, providing a much better visual experience for the player.



In this lesson, we will implement the enemy monster's attack logic. Currently, when it is an enemy's turn, the game only prints a message to the console. We will expand this functionality to have the enemy select a valid attack, play an animation, and apply damage to one of the player's monsters.

Choosing a Random Attack

The first step is to create a method that allows an enemy monster to select an attack. This logic should reside within the monster's resource script, as it needs access to the monster's available attacks and current energy points (EP). This method will select a random attack that the monster can actually afford to use.

Open the `monster_res.gd` script and add the following function:

```
func get_random_attack():
    var attacks = get_attacks()
    var available_attacks = []
    for attack in attacks:
        if Data.attack_data[attack]['cost'] <= current_ep:
            available_attacks.append(attack)
    return available_attacks.pick_random() if available_attacks.size() else null
```

Let's break down this new function:

- It first calls `get_attacks()` to retrieve all attacks the monster has unlocked based on its level.
- It initializes an empty array called `available_attacks`.
- It then iterates through all unlocked attacks. For each attack, it checks if the monster's `current_ep` is greater than or equal to the attack's cost.
- If the monster has enough energy, the attack is added to the `available_attacks` array.
- Finally, it returns a random attack from the `available_attacks` list. If the list is empty (meaning the monster cannot afford any of its attacks), it returns `null`.

Implementing the Attack in Battle

Now that we have a way for a monster to choose an attack, we can implement the attack sequence in the `battle.gd` script. Navigate to the `monster_ready` function. We will replace the existing `print("monster attack")` line with our new logic.

First, call the `get_random_attack` function and store the result in a variable.

```
func monster_ready(battle_sprite: Sprite2D):
    current_battle_sprite = battle_sprite
    if battle_sprite.playable:
        $BattleMenu.reveal(battle_sprite)
    else:
        var attack = battle_sprite.monster_res.get_random_attack()
        # More code will go here...
        battle_continue()
```

Next, we need to check if a valid attack was returned. A common mistake is to check an enum value directly in an if statement (e.g., `if attack:`). This can fail if the enum's integer value is 0, as if 0 evaluates to false. The correct way to check if the variable holds a valid enum member is to use the



is keyword.

Inside the if block, we will determine the target, play the animation, and apply the effects, much like we did for the player's attack.

1. Check if the attack is offensive to determine the target group. If it is, the target is the player's side; otherwise, it's the enemy's side.
2. Pick a random monster from the target group to be the target.
3. Play the attack effect animation at the target's position.
4. Play the attacking monster's own attack animation.
5. Calculate the damage amount based on the attack's power and the monster's level.
6. Apply the damage to the target by calling its `change_health` method.
7. Reduce the attacking monster's energy by the attack's cost.

Here is the complete code for the enemy's turn within the `monster_ready` function:

```
func monster_ready(battle_sprite: Sprite2D):
    current_battle_sprite = battle_sprite
    if battle_sprite.playable:
        $BattleMenu.reveal(battle_sprite, player_reserve_monsters)
    else:
        var attack = battle_sprite.monster_res.get_random_attack()
        if attack is Data.Attack:
            var offensive = Data.attack_data[attack]['offensive']
            var target_group = $BattleSprites/Player if offensive else $BattleSprites/Enemy
            var target = target_group.get_children().pick_random() as Sprite2D
            $EffectSprites/BattleAttackSprite.play(attack, target.position)
            current_battle_sprite.attack_animation()

            var amount = Data.attack_data[attack]['amount'] * current_battle_sprite.monster_res.level * 0.25
            target.change_health(amount, Data.attack_data[attack]['element'])
            current_battle_sprite.change_energy(-Data.attack_data[attack]['cost'])
            battle_continue()
```

With this implementation, enemy monsters will now actively participate in battles. They will choose an affordable attack at random, execute it against a random valid target, and deplete their energy accordingly, making the battle system fully functional for both sides.



With our attack system in place, we can now shift our focus to another core mechanic: catching monsters. This process will be relatively straightforward. The main steps involve checking an enemy monster's health, expanding our existing selection sprite logic to accommodate a "catch" state, and finally, adding the captured monster to our collection of monster resources.

Handling the Catch Menu Option

First, we need to handle the player's input when they select the "Catch" option from the battle menu. This logic resides within the `battle_menu.gd` script. When the state changes to `CATCH`, we need to emit the select signal to inform the main battle scene that the player is about to choose a target.

In the `state_handler` function within `battle_menu.gd`, add a case for the catch state:

```
func state_handler(new_state):
    state = new_state
    for node in get_children():
        node.hide()
    match state:
        Data.MenuState.MAIN:
            $MainSelection.show()
            main_button_setup()
        Data.MenuState.ATTACK:
            $AttackContainer.show()
            attack_button_setup()
        Data.MenuState.CATCH:
            select.emit(Data.MenuState.CATCH, null)
```

Activating the Selection Sprite

The select signal is received by the `_on_battle_menu_select` function in our main `battle.gd` script. We need to expand this function to handle the `CATCH` state. If the state is `CATCH`, we will activate the selection sprite, targeting the enemy monsters.

Update the `_on_battle_menu_select` function in `battle.gd` as follows:

```
func _on_battle_menu_select(state: int, info: Variant) -> void:
    if state == Data.MenuState.ATTACK:
        var attack_data = Data.attack_data[info]
        var target = $BattleSprites/Enemy if attack_data['offensive'] else $BattleSprites/P
        layer
        $EffectSprites/BattleSelectSprite.activate(target, info, Data.MenuState.ATTACK)
    if state == Data.MenuState.CATCH:
        var target = $BattleSprites/Enemy
        $EffectSprites/BattleSelectSprite.activate(target, info, Data.MenuState.CATCH)
```

Customizing the Selection Sprite

Currently, our selection sprite always shows a sword icon, which is appropriate for attacking but not for catching. We need to update it to show a different icon, such as a hand, when in the catch state. To do this, we will modify the `battle_select_sprite.gd` script.



First, create a constant dictionary to store the textures for each state. Then, in the activate function, use the incoming state to select the correct texture from this dictionary.

```
extends Sprite2D
```

```
var target: Node2D
var info
var state: Data.MenuState
var index: int
const textures = {
    Data.MenuState.ATTACK: preload("res://graphics/UI/options5.png"),
    Data.MenuState.CATCH: preload("res://graphics/UI/options8.png"),
}
```

```
signal cancel(state: Data.MenuState)
signal use(index: int, info, state: Data.MenuState)
```

```
func _ready() -> void:
    hide()
```

```
func activate(new_target, new_info, new_state: Data.MenuState):
    show()
    target = new_target
    info = new_info
    state = new_state
    position = target.get_child(index).position
    texture = textures[state]
```

Implementing Cancellation

If the player decides not to catch a monster after entering the selection mode, they should be able to press a cancel button to return to the main battle menu. The BattleSelectSprite already emits a cancel signal, which is handled in battle.gd and passed to the cancel function in battle_menu.gd. We just need to update this function to handle the CATCH state.

In battle_menu.gd, modify the cancel function:

```
func cancel(new_state: Data.MenuState):
    if new_state == Data.MenuState.ATTACK:
        state = Data.MenuState.ATTACK
    if new_state == Data.MenuState.CATCH:
        state = Data.MenuState.MAIN
    $Support/StopTimer.start()
```

Implementing the Catch Logic

The final step is to define what happens when the player confirms their target. This is handled in the `_on_battle_select_sprite_use` function in battle.gd. We will add logic to check if the selected monster's health is low enough to be caught.

A monster can only be caught if its current HP is below a certain percentage of its maximum HP. For now, we will set this threshold to 90% for easier testing, but you can adjust it to a lower value like



10% or 20% for the final game. If the catch is successful, we will add the monster's resource to our collection and remove the monster from the battlefield. However, we need additional logic for this, so for now, we'll just log a print statement for testing purposes.

Update the `_on_battle_select_sprite_use` function in `battle.gd`:

```
func _on_battle_select_sprite_use(index: int, info: Variant, state: int) -> void:
    if state == Data.MenuState.ATTACK:
        # ... conditional code ...
    if state == Data.MenuState.CATCH:
        var target = $BattleSprites/Enemy.get_child(index)
        if target.monster_res.current_hp <= target.monster_res.get_stat('max hp') * 0.9:
            print('catch monster', target.monster_res)
            target.queue_free()
        battle_continue()
```

With these changes, the core logic for catching monsters is now implemented. You can test it by attacking an enemy monster to lower its health and then attempting to catch it. The system for storing and managing caught monsters will be fully integrated after we implement the monster swapping feature in a future lesson.



In this lesson, we will implement one of the final major mechanics for our game: swapping monsters during a battle. This involves managing monsters that are not actively fighting (the reserve), creating a user interface for the player to select a monster from the reserve, and updating the battle state accordingly.

Expanding the Monster Roster

To properly test a swapping system, the player needs more than the three monsters that can be on the field at once. We will begin by expanding the player's monster list.

Navigate to the data.gd script. Locate the `player_monsters` array and add more monsters to it. For this example, we will increase the total to six, but you can add any monsters you prefer.

```
var player_monsters = [  
    new_monster_res(Monster.SPARCHU, 60),  
    new_monster_res(Monster.ATROX, 10),  
    new_monster_res(Monster.GULFIN, 50),  
    new_monster_res(Monster.CLEAF, 50),  
    new_monster_res(Monster.FINSTA, 50),  
    new_monster_res(Monster.PLUMETTE, 50),  
]
```

Managing Reserve Monsters

With more monsters available, we need a way to separate the active ones from the reserves. In the `battle.gd` script, we will create two new array variables to hold the reserve monsters for both the player and the enemy.

```
extends Node2D
```

```
var battle_sprite_scene = preload("res://scenes/battle/battle_sprite.tscn")  
var current_battle_sprite: Sprite2D  
var player_reserve_monsters: Array  
var enemy_reserve_monsters: Array
```

Next, we will populate these arrays. In the `battle_sprite_setup` function, after setting up the active monsters, we can use the `slice` method on the `data` array. This method returns a new array containing elements from a given starting index. Since the first three monsters (indices 0, 1, and 2) are active, our reserve will consist of any monsters from index 3 onwards.

We will add this logic for both the player and the enemy.

```
func battle_sprite_setup(start_positions: Node2D, parent: Node2D, is_player: bool, data: Array):  
    for start_pos_index: int in start_positions.get_child_count():  
        if start_pos_index < data.size():  
            var pos = start_positions.get_child(start_pos_index).position  
            var monster_res: MonsterResource = data[start_pos_index]  
            create_battle_sprite(pos, parent, is_player, monster_res, start_pos_index)  
        if is_player:  
            player_reserve_monsters = data.slice(3)
```

```
else:  
    enemy_reserve_monsters = data.slice(3)
```

Tracking Monster Position Index

When we swap a monster, the new monster needs to appear in the correct position on the battlefield. To facilitate this, we must store the initial position index (0, 1, or 2) within each BattleSprite instance.

1. First, update the call to `create_battle_sprite` inside `battle_sprite_setup` in `battle.gd` to pass the `start_pos_index`.
2. Next, modify the `create_battle_sprite` function signature to accept this new integer parameter, which we'll call `index`.
3. Pass this index to the battle sprite's setup function.

Your updated functions in `battle.gd` should look like this:

```
func battle_sprite_setup(start_positions: Node2D, parent: Node2D, is_player: bool, data: Array):  
    for start_pos_index: int in start_positions.get_child_count():  
        if start_pos_index < data.size():  
            var pos = start_positions.get_child(start_pos_index).position  
            var monster_res: MonsterResource = data[start_pos_index]  
            create_battle_sprite(pos, parent, is_player, monster_res, start_pos_index)  
    if is_player:  
        player_reserve_monsters = data.slice(3)  
    else:  
        enemy_reserve_monsters = data.slice(3)
```

```
func create_battle_sprite(pos: Vector2, parent: Node2D, is_player: bool, res: MonsterResource, index: int):  
    var battle_sprite = battle_sprite_scene.instantiate()  
    battle_sprite.setup(pos, is_player, res, index)  
    parent.add_child(battle_sprite)  
    battle_sprite.connect("is_ready", monster_ready)
```

Now, open `battle_sprite.gd`. We need to add a variable to store the index and update the setup function to receive and assign it.

```
extends Sprite2D  
var monster_res: MonsterResource  
var paused: bool  
var readiness: float  
var playable: bool  
var index: int  
  
signal is_ready(battle_sprite: Sprite2D)  
  
# ...
```



```
func setup(pos: Vector2, is_player: bool, res: MonsterResource, new_index: int):  
    position = pos  
    flip_h = is_player  
    playable = is_player  
    monster_res = res  
    index = new_index  
    texture = load(Data.monster_data[res.id]['battle texture'])  
    ui_setup()
```

Creating the Swap Menu

With the data structure in place, we can build the UI. In the BattleMenu scene, we will add a new container for our swap menu. The structure will be a PanelContainer named SwapContainer, containing a ScrollContainer, which in turn contains a VBoxContainer. This structure ensures that if the player has many reserve monsters, the list will be scrollable.

- SwapContainer (PanelContainer)
 - ScrollContainer (ScrollContainer)
 - VBoxContainer (VBoxContainer)

Configure the SwapContainer to have a custom minimum size of 90px on the X and 100 on the Y. Then, change the Anchor Offsets so Left is 40px. Ensure the VBoxContainer is set to expand vertically and horizontally to fill the ScrollContainer.

The Swap Button Scene

Each monster in the reserve will be represented by a button. Let's create a new scene for this button.

1. Create a new scene with a Button node as the root. Name it BattleSwapButton.
2. Add an HBoxContainer as a child of the button.
3. Add a TextureRect and a Label as children of the HBoxContainer. You can fill this out with any placeholder data, as we'll update these buttons dynamically via script.
4. Configure the button's Layout to have a Vertical `Size Flag` of `Expand Fill`. Set a custom minimum height (e.g., 30 pixels).
5. Save the scene as battle_swap_button.tscn.

Now, attach a script to the BattleSwapButton root node. This script will have a setup function to populate the button's label and texture with the correct monster data.

```
extends Button
```

```
var res: MonsterResource  
var index: int
```

```
func setup(monster_res: MonsterResource, new_index: int):  
    res = monster_res  
    index = new_index  
    $HBoxContainer/Label.text = Data.monster_data[res.id]['name']  
    $HBoxContainer/TextureRect.texture = load(Data.monster_data[res.id]['icon texture'])
```

Populating the Swap Menu



Back in `battle_menu.gd`, we need to dynamically create and add these buttons to our `VBoxContainer` when the player chooses to swap.

First, let's handle the `SWAP` menu state in the `state_handler` function. It should show the `SwapContainer` and call a new function, `swap_button_setup`.

```
func state_handler(new_state):
    state = new_state
    for node in get_children():
        node.hide()
    match state:
        Data.MenuState.MAIN:
            # ...
        Data.MenuState.ATTACK:
            # ...
        Data.MenuState.CATCH:
            # ...
        Data.MenuState.SWAP:
            $SwapContainer.show()
            swap_button_setup()
```

The `swap_button_setup` function needs access to the player's reserve monsters. We must pass this data from `battle.gd` to `battle_menu.gd`. In `battle.gd`, update the call to `$BattleMenu.reveal` inside the `monster_ready` function.

```
func monster_ready(battle_sprite: Sprite2D):
    current_battle_sprite = battle_sprite
    if battle_sprite.playable:
        $BattleMenu.reveal(battle_sprite, player_reserve_monsters)
    # ...
```

In `battle_menu.gd`, update the `reveal` function to accept this new array and store it in a class variable, which we'll call `monster_list`.

```
var monster_list: Array
```

```
# ...
```

```
func reveal(battle_sprite: Sprite2D, reserve_monsters: Array):
    show()
    state = Data.MenuState.MAIN
    position = battle_sprite.position
    monster_res = battle_sprite.monster_res
    monster_list = reserve_monsters
```

Finally, we can implement `swap_button_setup`. This function will clear any old buttons, loop through the `monster_list`, and for each monster, it will instantiate our `BattleSwapButton` scene, call its `setup` function, and add it to the `VBoxContainer`. After adding all buttons, it will grab focus on the first one to enable keyboard/controller navigation. Note that we will preload the button scene so we can



instantiate it.

```
var battle_swap_button_scene = preload("res://scenes/battle/battle_swap_button.tscn")
```

```
func swap_button_setup():  
    for btn in $SwapContainer/ScrollContainer/VBoxContainer.get_children():  
        btn.queue_free()  
    for i in monster_list.size():  
        var btn = battle_swap_button_scene.instantiate() as Button  
        var res = monster_list[i]  
        btn.setup(res, i)  
        $SwapContainer/ScrollContainer/VBoxContainer.add_child(btn)  
    await get_tree().process_frame  
    $SwapContainer/ScrollContainer/VBoxContainer.get_child(0).grab_focus()
```

With these changes, running the game and selecting the “Swap” option in a battle will now display a list of your reserve monsters. You can navigate the list, but pressing the buttons will not have an effect just yet. We will implement the swapping logic in a future lesson.



In this lesson, we will complete the implementation of the monster swapping mechanic. This will involve creating and connecting signals between our various battle scripts to allow the player to switch their active monster with one from their reserve list.

The Swap Button

First, we need to modify the `battle_swap_button.gd` script. When a player clicks a swap button, it should communicate which monster was selected. We can achieve this by emitting a signal containing the necessary information.

We will define a press signal that sends the menu state and an array containing the monster's index and its resource. The `_on_pressed` function will then emit this signal with the correct data.

Update your `battle_swap_button.gd` script with the following code:

```
extends Button

var res: MonsterResource
var index: int
signal press(state: Data.MenuState, info)

func setup(monster_res: MonsterResource, new_index: int):
    res = monster_res
    index = new_index
    $HBoxContainer/Label.text = Data.monster_data[res.id]['name']
    $HBoxContainer/TextureRect.texture = load(Data.monster_data[res.id]['icon texture'])

func _on_pressed() -> void:
    press.emit(Data.MenuState.SWAP, [index, res])
```

Updating the Battle Menu

Next, we need to update the `battle_menu.gd` script to handle the signal from our swap button. The battle menu will act as an intermediary, receiving the button press and then emitting a more specific swap signal to the main battle scene.

1. Define a new swap signal that will carry the monster's index and resource.
2. In the `button_handler` function, add a case for `Data.MenuState.SWAP`. When this state is received, it will emit our new swap signal with the information it received from the button.
3. Ensure that in the `swap_button_setup` function, the press signal from each instantiated swap button is connected to the `button_handler`.

Here are the necessary additions to `battle_menu.gd`:

```
extends Control
# ... other variables

signal defend
signal select(state: Data.MenuState, info)
signal swap(index: int, res: MonsterResource)
```



```
# ... _ready() and _input() functions

func button_handler(current_state: Data.MenuState, info):
    match current_state:
        Data.MenuState.MAIN:
            state = info
        Data.MenuState.ATTACK:
            state = Data.MenuState.SELECT
            select.emit(Data.MenuState.ATTACK, info)
        Data.MenuState.SWAP:
            swap.emit(info[0], info[1])
            finish()
    if current_state == Data.MenuState.MAIN and info == Data.MenuState.DEFEND:
        defend.emit()

# ... main_button_setup() and attack_button_setup() functions

func swap_button_setup():
    for btn in $SwapContainer/ScrollContainer/VBoxContainer.get_children():
        btn.queue_free()
    for i in monster_list.size():
        var btn = battle_swap_button_scene.instantiate() as Button
        var res = monster_list[i]
        btn.setup(res, i)
        btn.connect('press', button_handler)
        $SwapContainer/ScrollContainer/VBoxContainer.add_child(btn)
    await get_tree().process_frame
    $SwapContainer/ScrollContainer/VBoxContainer.get_child(0).grab_focus()

# ... other functions
```

Handling the Swap in the Battle Scene

Now we need to implement the core swap logic in the main battle.gd script. This involves connecting to the swap signal from the BattleMenu and defining the function that will execute the swap.

The logic for swapping is as follows:

- Remove the chosen monster from the player_reserve_monsters list.
- Add the monster that is currently in battle back into the reserve list.
- Update the active battle sprite to display the new monster.
- Continue the battle sequence.

You will need to connect the swap signal from the BattleMenu node to a new function in the Battle script, which can be done through the Godot editor's Node tab. Then, add the following function to battle.gd:

```
func _on_battle_menu_swap(index: int, res: MonsterResource) -> void:
    player_reserve_monsters.remove_at(index)
    player_reserve_monsters.append(current_battle_sprite.monster_res)
    current_battle_sprite.change(res)
```



```
battle_continue()
```

Updating the Battle Sprite

The final step is to create the change function within the `battle_sprite.gd` script. This function is responsible for visually updating the monster on the battlefield. It will change the sprite's texture, update its internal monster resource, and refresh its UI to reflect the new monster's stats.

Add this new function to your `battle_sprite.gd` script:

```
func change(res: MonsterResource):  
    texture = load(Data.monster_data[res.id]['battle texture'])  
    monster_res = res  
    ui_setup()
```

With these changes, the swap mechanic is now fully functional. When you run the game, you can enter the swap menu, select a monster from your reserve, and it will switch places with the monster currently on the field, allowing the battle to continue with your new active monster.



In this lesson, we will implement the death mechanic for our battle system. When a monster's health reaches zero, it should be removed from the battle. If the monster's owner has reserve monsters, a new one will take its place. This mechanic is crucial for determining the outcome of a battle: if all of the player's monsters are defeated, it's game over, and if all enemy monsters are defeated, the player wins.

Checking for Monster Death

The first step is to detect when a monster's health drops to or below zero. We will handle this within the `battle_sprite.gd` script, which controls each individual monster in the battle.

We need a way to signal to the main battle scene that a monster has been defeated. For this, we will define a new signal called `death`. This signal will pass two arguments: a boolean to indicate if the monster belonged to the player, and an integer for its index on the battlefield.

In `battle_sprite.gd`, add the signal definition at the top of the script:

```
signal death(is_player: bool, index: int)
```

Next, we need a function to check the monster's health. We will create a new function called `check_death`. This function will verify if the monster's `current_hp` is zero or less. If it is, we will emit our new `death` signal and then remove the monster's node from the scene tree using `queue_free()`.

Add the `check_death` function to `battle_sprite.gd`:

```
func check_death():
    if monster_res.current_hp <= 0:
        death.emit(playable, index)
        queue_free()
```

Finally, we need to call this check at the appropriate time. In the existing `change_health` function, a `tween` animates the health bar. We can add a callback to this `tween` that executes our `check_death` function once the animation is complete.

Modify the `change_health` function in `battle_sprite.gd`:

```
func change_health(amount: float, element: Data.Element):
    var modifier = Data.element_modifier[element][monster_res.get_element()]
    monster_res.current_hp -= amount * modifier
    var tween = create_tween()
    tween.tween_property(
        $Control/StatsContainers/HealthBar,
        "value",
        monster_res.current_hp,
        0.5)
    tween.tween_callback(check_death)
```

Handling Monster Death



Now that our BattleSprite can signal its own defeat, the main battle.gd script needs to listen for this signal and react accordingly. The main task is to replace the defeated monster with one from the reserve list, if available.

First, we must connect to the death signal whenever a new BattleSprite is created. In battle.gd, update the create_battle_sprite function to include the connection.

```
func create_battle_sprite(pos: Vector2, parent: Node2D, is_player: bool, res: Monster
Resource, index: int):
    var battle_sprite = battle_sprite_scene.instantiate()
    battle_sprite.setup(pos, is_player, res, index)
    parent.add_child(battle_sprite)
    battle_sprite.connect("is_ready", monster_ready)
    battle_sprite.connect('death', monster_death)
```

Next, we will create the monster_death function that is called by the signal. This function will contain the logic for replacing a monster. It will:

- Determine which reserve list to use (player or enemy) based on the is_player parameter.
- Check if there are any monsters in that reserve list.
- If so, it gathers the necessary information (position, parent node, and the new monster resource) and calls create_battle_sprite to bring the new monster into the battle.

Add the monster_death function to battle.gd:

```
func monster_death(is_player: bool, index: int):
    var reserve_list = player_reserve_monsters if is_player else enemy_reserve_monsters
    var start_positions = $StartPositions/Player if is_player else $StartPositions/Enemy
    var pos = start_positions.get_child(index).position
    var parent = $BattleSprites/Player if is_player else $BattleSprites/Enemy
    if reserve_list.size():
        create_battle_sprite(pos, parent, is_player, reserve_list.pop_at(0), index)
```

Testing the Replacement

To test this new system, we need to ensure there are monsters in the reserve lists. Let's modify the enemy_monsters array in data.gd to include a reserve monster. This will allow us to see the replacement mechanic in action when an enemy is defeated.

In data.gd, update the enemy_monsters variable:

```
var enemy_monsters = [
    new_monster_res(Monster.CINDRILL, 18),
    new_monster_res(Monster.POUCH, 19),
    new_monster_res(Monster.JACANA, 17),
    new_monster_res(Monster.LARVEA, 17),
]
```

With this change, the first three monsters will appear on the battlefield, and Larvea will be held in



reserve. When one of the active enemy monsters is defeated, Larvea should appear in its place.

Implementing Win and Loss Conditions

The final piece of the death mechanic is to check for victory or defeat after a monster is defeated. A condition is met when a side has no reserve monsters left and their last active monster is defeated.

We will add this logic to the end of our `monster_death` function in `battle.gd`.

- **Loss Condition:** If the player's reserve list is empty and the number of active player monsters is one (the one that just triggered the `monster_death` function), the game will quit.
- **Win Condition:** If the enemy's reserve list is empty and they are down to their last monster, we will call a `finish_battle` function.

First, create a placeholder `finish_battle` function in `battle.gd`.

```
func finish_battle():  
    print('battle finished')
```

Now, update the `monster_death` function with the win/loss checks:

```
func monster_death(is_player: bool, index: int):  
    var reserve_list = player_reserve_monsters if is_player else enemy_reserve_monsters  
    var start_positions = $StartPositions/Player if is_player else $StartPositions/Enemy  
    var pos = start_positions.get_child(index).position  
    var parent = $BattleSprites/Player if is_player else $BattleSprites/Enemy  
    if reserve_list.size():  
        create_battle_sprite(pos, parent, is_player, reserve_list.pop_at(0), index)  
  
    if player_reserve_monsters.size() == 0 and $BattleSprites/Player.get_child_count() == 1:  
        get_tree().quit()  
  
    if enemy_reserve_monsters.size() == 0 and $BattleSprites/Enemy.get_child_count() == 1:  
        finish_battle()
```

Fixing a Selection Bug

During testing, a bug may appear. After a monster is defeated, the number of available targets changes. If the player then tries to select a target, the selection sprite might point to an invalid index, causing a crash. To fix this, we need to reset the selection index to 0 every time the selection sprite is activated.

In `battle_select_sprite.gd`, modify the `activate` function to reset the index variable.

```
func activate(new_target, new_info, new_state: Data.MenuState):  
    show()  
    target = new_target  
    info = new_info  
    state = new_state
```



```
index = 0
position = target.get_child(index).position
texture = textures[state]
```

With this fix, the selection cursor will always start on the first available target, preventing errors. You can now test the full battle loop by defeating all enemy monsters to see the “battle finished” message appear in the output log.



In this lesson, we will add the final polishing touches to our battle system. We will implement passive energy recovery, disable attack buttons when a monster lacks sufficient energy, significantly improve the UI styling, and implement the monster-catching mechanic. These changes will complete the core functionality and visual appeal of our battles.

Energy Recovery During Battle

To make battles more dynamic, monsters should slowly recover energy over time. This can be implemented within the `\_process` function of the `BattleSprite` script. We will add a line to increase the monster's energy on each frame, provided the battle is not paused.

Open the `battle_sprite.gd` script and modify the `_process` function as follows:

```
func _process(delta: float) -> void:
    if not paused:
        readiness += monster_res.get_stat('speed') * delta
        $Control/ReadyProgressBar.value = readiness
        change_energy(monster_res.get_stat('max ep') * delta * 0.01)
```

Here, we call the `change_energy` function, increasing the energy by a small fraction of the monster's maximum energy points (EP) multiplied by `delta`. The 0.01 multiplier ensures the recovery is slow and balanced.

Disabling Attacks with Insufficient Energy

A monster should not be able to use an attack if it does not have enough energy. To enforce this, we will disable the corresponding attack button. This requires passing the monster's current energy to the button when it's created.

1. First, update the `setup` function in `battle_attack_button.gd` to accept the monster's current EP and set the button's disabled property if the energy is too low.

```
func setup(attack_enum: Data.Attack, current_ep: float):
    text = Data.attack_data[attack_enum]['name']
    info = attack_enum
    if current_ep < Data.attack_data[attack_enum]['cost']:
        disabled = true
```

2. Next, in `battle_menu.gd`, modify the `attack_button_setup` function to pass the monster's current energy when setting up each button.

```
func attack_button_setup():
    for button in $AttackContainer.get_children():
        button.queue_free()
    for attack_enum in monster_res.get_attacks():
        var btn = battle_attack_button_scene.instantiate()
        btn.setup(attack_enum, monster_res.current_ep)
        btn.connect('press', button_handler)
        $AttackContainer.add_child(btn)
    await get_tree().process_frame
    $AttackContainer.get_child(0).grab_focus()
```

Now, if a monster's `current_ep` is less than an attack's cost, that attack's button will be grayed out and unclickable.

Improving the UI with Theme Overrides

Our current UI is functional but lacks visual appeal. We can greatly improve it by using Godot's Theme Override options in the Inspector.

Styling the Attack Buttons

Let's style the `battle_attack_button.tscn` scene.

1. Select the root **Button** node.
2. In the Inspector, expand the **Theme Overrides** section.
3. Expand **Styles**. For the **Normal**, **Focus**, **Disabled**, and **Pressed** states, we will create a new **StyleBoxTexture**.
4. For each state, assign the appropriate texture from the `res://graphics/UI/` folder (e.g., `frame.png` for Normal, `frame_selected.png` for Focus, `frame_disabled.png` for Disabled).
5. After assigning a texture, expand the **StyleBoxTexture** properties and set the **Texture Margins** for Top, Right, Bottom, and Left to 5. This prevents the texture's border from stretching.
6. Expand **Colors** under Theme Overrides. Set the **Font Color** for the Normal, Focus, and Disabled states to appropriate colors (e.g., a dark color for normal/focus, and a reddish color for disabled).
7. Expand **Fonts** and drag your desired font, such as `dogicapixel.otf`, into the **Font** property.

Styling the Monster UI

The monster's nameplate in the `battle_sprite.tscn` scene also needs styling.

- Select the **NameContainer** node.
- Under **Theme Overrides > Styles**, create a new **StyleBoxTexture** for the **Panel** property.
- Use `frame.png` as the texture and set the **Texture Margins** to 5 on all sides.
- Select the **Label** node inside the NameContainer.
- Under **Theme Overrides > Colors**, set the **Font Color** to a dark color.
- Under **Theme Overrides > Fonts**, assign your custom font.

Styling the Swap Menu

The swap menu requires several adjustments to look good.

1. In `battle_menu.tscn`, select the **SwapContainer**. Under **Theme Overrides > Styles**, set the **Panel** property to a new **StyleBoxEmpty**. This will make the container's background transparent.
2. Open the `battle_swap_button.tscn` scene and style the root **Button** node using the same **Theme Overrides** techniques as the attack button (using `frame.png` for Normal and `frame_selected.png` for Focus). Also, style the Label's font and color as we've done before.
3. Back in `battle_menu.tscn`, select the **SwapContainer** and increase its **Size X** property to around 180 to give the buttons more space.
4. Select the **ScrollContainer** inside the SwapContainer. In the Inspector, set the **Vertical Scroll Mode** to **Show Never** to hide the scrollbar while keeping the scrolling functionality.
5. To add some padding on the left, add a new **Control** node to the **VBoxContainer** as its first child. Give it a **Custom Minimum Size** with an X value of 4.

Implementing the Catch Mechanic



Finally, let's implement the logic for catching a monster. Previously, we only printed a message. Now, we will add the caught monster to the player's reserve.

In the battle.gd script, find the `_on_battle_select_sprite_use` function. Update the logic for the CATCH state.

```
func _on_battle_select_sprite_use(index: int, info: Variant, state: int) -> void:
    if state == Data.MenuState.ATTACK:
        # ... attack logic remains the same
    if state == Data.MenuState.CATCH:
        var target = $BattleSprites/Enemy.get_child(index)
        if target.monster_res.current_hp <= target.monster_res.get_stat('max hp') * 0.9:
            player_reserve_monsters.append(target.monster_res)

            target.queue_free()
        battle_continue()
```

This code now checks if the target monster's health is sufficiently low. If the catch is successful, it appends the monster's resource to the `player_reserve_monsters` array and removes the monster from the battlefield using `queue_free()`. With this, our battle system is now feature-complete.



Congratulations on completing this extensive part of the course! You have successfully created a feature-rich, turn-based fighting game from the ground up. This was a significant undertaking that involved implementing a wide array of game mechanics and systems. Well done for your dedication and for making it this far.

Course Recap: What You Have Accomplished

Throughout this course, we have built a solid foundation for our monster-fighting game. You have learned to implement a variety of core features and now have a functional battle system. The key mechanics you have created include:

- Monster attacks, damage calculation, and energy consumption.
- The ability to swap between monsters during a battle.
- A system for catching new monsters.
- Animations to bring the battles to life.
- Various menus for user interaction.
- Management of complex game data.

What's Next: Creating the Final Game

With the battle system complete, we will now move on to the final and most exciting stage: integrating our work into a complete game world. In the next part of this series, we will combine the overworld exploration with the turn-based battle system to create a seamless and engaging player experience. Additionally, we will be adding several new features to round out the game.

Upcoming features will include:

- Combining the overworld with the battle system.
- A quest system to guide the player.
- Nurses for healing your monsters.
- Grass patches to trigger encounters with wild monsters.
- A level-up mechanic for monster progression.

We look forward to seeing you in the next section as we build the final game!

Learning with Zenva

Zenva is an education platform serving over a million students, offering a wide range of courses for both beginners and experienced developers looking to learn new skills. Our courses are designed to be versatile, allowing you to learn in the way that suits you best. You can:

- Watch the video tutorials.
- Read the detailed lesson summaries.
- Follow along with the instructor using the included project files.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

data.gd

Found in folder: **/global**

This code defines various constants and variables for an overworld and battle system in a game. It includes enums for character styles, locations, monsters, elements, attacks, menu states, and biomes. It also defines dictionaries for character view directions, character textures, level paths, monster data, attack data, element modifiers, and biome backgrounds. The code also initializes player and enemy monsters and sets the current biome and trainer fight flag.

extends Node

```
# overworld data
enum CharacterStyle {CHARACTER, BLOND, PLAYER, GREEN, FIRE, GRASS, ICE, PURPLE, STRAW, BOY, GIRL}
enum Location {OVERWORLD, HOUSE, HOSPITAL, GRASS_ARENA, FIRE_ARENA}
const ANIMATION_SPEED = 6
const TILE_ANIMATION_SPEED = 2
const character_view_directions = {
    Vector2i.DOWN: 0,
    Vector2i.LEFT: 1,
    Vector2i.RIGHT: 2,
    Vector2i.UP: 3,
    Vector2i.ZERO: 0,
    Vector2i(1,1) : 2, # down right
    Vector2i(1,-1) : 2, # up right
    Vector2i(-1,1) : 1, # down left
    Vector2i(-1,-1) : 1, # up left
}
const character_texture_data = {
    CharacterStyle.PLAYER: "res://graphics/characters/player.png",
    CharacterStyle.GREEN: "res://graphics/characters/green.png",
    CharacterStyle.BLOND: "res://graphics/characters/blond.png",
    CharacterStyle.CHARACTER: "res://graphics/characters/character.png",
    CharacterStyle.FIRE: "res://graphics/characters/fire_boss.png",
    CharacterStyle.GRASS: "res://graphics/characters/grass_boss.png",
    CharacterStyle.ICE: "res://graphics/characters/ice_boss.png",
    CharacterStyle.STRAW: "res://graphics/characters/straw.png",
    CharacterStyle.PURPLE: "res://graphics/characters/purple.png",
    CharacterStyle.BOY: "res://graphics/characters/young_boy.png",
    CharacterStyle.GIRL: "res://graphics/characters/young_girl.png",
}
const LEVEL_PATHS = {
    Location.OVERWORLD: "res://scenes/levels/overworld.tscn",
    Location.HOUSE: "res://scenes/levels/house.tscn",
    Location.HOSPITAL: "res://scenes/levels/hospital.tscn",
    Location.GRASS_ARENA: "res://scenes/levels/plant_arena.tscn",
    Location.FIRE_ARENA: "res://scenes/levels/fire_arena.tscn"}
```



```
var current_char: Character
var current_loc: Data.Location
var char_data = {}

# battle data
enum Monster {SPARCHU, ATROX, JACANA, CINDRILL, CLEAF, DRAEM, FINIETTE, FINSTA, FRIOL
ERA, GULFIN, IVIERON, LARVEA, PLUMA, PLUMETTE, POUCH, VULKEO}
enum Element {FIRE, WATER, PLANT}
enum Attack {CLAW, FIRE, ICE, WATER, HEAL, EXPLOSION}
enum MenuState {MAIN, ATTACK, DEFEND, SWAP, CATCH, SELECT}
enum Biome {GRASS, DESERT, ICE}
const monster_data = {
  Monster.SPARCHU: {
    'name': 'Sparchu',
    'battle texture': "res://graphics/battle sprites/Sparchu.png",
    'icon texture': "res://graphics/menu sprites/Sparchu.png",
    'stats': {'max hp': 10, 'max ep': 2, 'speed': 1},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.FIRE},
  Monster.CINDRILL: {
    'name': 'Cindrill',
    'battle texture': "res://graphics/battle sprites/Cindrill.png",
    'icon texture': "res://graphics/menu sprites/Cindrill.png",
    'stats': {'max hp': 14, 'max ep': 12, 'speed': 1},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.FIRE},
  Monster.VULKEO: {
    'name': 'Vulkeo',
    'battle texture': "res://graphics/battle sprites/Vulkeo.png",
    'icon texture': "res://graphics/menu sprites/Vulkeo.png",
    'stats': {'max hp': 17, 'max ep': 14, 'speed': 1},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.FIRE},
  Monster.FINSTA: {
    'name': 'Finsta',
    'battle texture': "res://graphics/battle sprites/Finsta.png",
    'icon texture': "res://graphics/menu sprites/Finsta.png",
    'stats': {'max hp': 5, 'max ep': 6, 'speed': 0.7},
    'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
    'element': Element.WATER},
  Monster.GULFIN: {
    'name': 'Gulfin',
    'battle texture': "res://graphics/battle sprites/Gulfin.png",
    'icon texture': "res://graphics/menu sprites/Gulfin.png",
    'stats': {'max hp': 9, 'max ep': 8, 'speed': 0.8},
    'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
    'element': Element.WATER},
  Monster.FINIETTE: {
    'name': 'Finiette',
    'battle texture': "res://graphics/battle sprites/Finiette.png",
    'icon texture': "res://graphics/menu sprites/Finiette.png",
    'stats': {'max hp': 11, 'max ep': 10, 'speed': 0.9},
```

```
'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
'element': Element.WATER},
Monster.PLUMETTE: {
  'name': 'Plumette',
  'battle texture': "res://graphics/battle sprites/Plumette.png",
  'icon texture': "res://graphics/menu sprites/Plumette.png",
  'stats': {'max hp': 6, 'max ep': 7, 'speed': 0.6},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
  'element': Element.PLANT},
Monster.IVIERON: {
  'name': 'Pluma',
  'battle texture': "res://graphics/battle sprites/Pluma.png",
  'icon texture': "res://graphics/menu sprites/Pluma.png",
  'stats': {'max hp': 8, 'max ep': 8, 'speed': 0.9},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION
},
  'element': Element.PLANT},
Monster.PLUMA: {
  'name': 'Pluma',
  'battle texture': "res://graphics/battle sprites/Pluma.png",
  'icon texture': "res://graphics/menu sprites/Pluma.png",
  'stats': {'max hp': 12, 'max ep': 9, 'speed': 1.1},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION
},
  'element': Element.PLANT},
Monster.LARVEA: {
  'name': 'Larvea',
  'battle texture': "res://graphics/battle sprites/Larvea.png",
  'icon texture': "res://graphics/menu sprites/Larvea.png",
  'stats': {'max hp': 6, 'max ep': 7, 'speed': 0.6},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION
},
  'element': Element.PLANT},
Monster.CLEAF: {
  'name': 'Pluma',
  'battle texture': "res://graphics/battle sprites/Pluma.png",
  'icon texture': "res://graphics/menu sprites/Pluma.png",
  'stats': {'max hp': 9, 'max ep': 8, 'speed': 1},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION
},
  'element': Element.PLANT},
Monster.DRAEM: {
  'name': 'Draem',
  'battle texture': "res://graphics/battle sprites/Draem.png",
  'icon texture': "res://graphics/menu sprites/Draem.png",
  'stats': {'max hp': 12, 'max ep': 9, 'speed': 1.1},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION
},
  'element': Element.PLANT},
Monster.POUCH: {
  'name': 'Pouch',
  'battle texture': "res://graphics/battle sprites/Pouch.png",
  'icon texture': "res://graphics/menu sprites/Pouch.png",
  'stats': {'max hp': 14, 'max ep': 3, 'speed': 0.5},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.EXPLOSION
```

```
,
  'element': Element.PLANT},
Monster.ATROX: {
  'name': 'Atrox',
  'battle texture': "res://graphics/battle sprites/Atrox.png",
  'icon texture': "res://graphics/menu sprites/Atrox.png",
  'stats': {'max hp': 12, 'max ep': 8, 'speed': 2},
  'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
},
  'element': Element.FIRE},
Monster.FRIOLERA: {
  'name': 'Friolera',
  'battle texture': "res://graphics/battle sprites/Friolera.png",
  'icon texture': "res://graphics/menu sprites/Friolera.png",
  'stats': {'max hp': 11, 'max ep': 10, 'speed': 0.9},
  'attacks': {0: Attack.CLAW, 12: Attack.WATER, 24: Attack.HEAL, 30: Attack.ICE},
  'element': Element.WATER},
Monster.JACANA: {
  'name': 'Jacana',
  'battle texture': "res://graphics/battle sprites/Jacana.png",
  'icon texture': "res://graphics/menu sprites/Jacana.png",
  'stats': {'max hp': 3, 'max ep': 5, 'speed': 4},
  'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
},
  'element': Element.FIRE}}
const attack_data = {
  Attack.CLAW: {
    'name': 'Claw',
    'offensive': true,
    'cost': 50,
    'amount': 10,
    'element': Element.PLANT,
    'texture': "res://graphics/attack effects/attack7.png",
    'sound': "res://audio/scratch.mp3"},
  Attack.FIRE: {
    'name': 'Fire',
    'offensive': true,
    'cost': 12,
    'amount': 15,
    'element': Element.FIRE,
    'texture': "res://graphics/attack effects/attack5.png",
    'sound': "res://audio/fire.wav"},
  Attack.WATER: {
    'name': 'Splash',
    'offensive': true,
    'cost': 9,
    'amount': 11,
    'element': Element.WATER,
    'texture': "res://graphics/attack effects/attack3.png",
    'sound': "res://audio/splash.wav"},
  Attack.ICE: {
    'name': 'Ice shards',
    'offensive': true,
    'cost': 13,
    'amount': 20,
```

```
'element': Element.WATER,
'texture': "res://graphics/attack effects/attack1.png",
'sound': "res://audio/ice.mp3"},
Attack.EXPLOSION: {
  'name': 'Explode',
  'offensive': true,
  'cost': 20,
  'amount': 100,
  'element': Element.FIRE,
  'texture': "res://graphics/attack effects/attack4.png",
  'sound': "res://audio/explosion.mp3"},
Attack.HEAL: {
  'name': 'Heal',
  'offensive': false,
  'cost': 15,
  'amount': -30,
  'element': Element.PLANT,
  'texture': "res://graphics/attack effects/attack6.png",
  'sound': "res://audio/green.wav"}}
const element_modifier = {
  Element.FIRE: {Element.FIRE: 1, Element.WATER: 0.5, Element.PLANT: 2},
  Element.WATER: {Element.FIRE: 2, Element.WATER: 1, Element.PLANT: 0.5},
  Element.PLANT: {Element.FIRE: 0.5, Element.WATER: 2, Element.PLANT: 1}}
const biome_bg = {
  Biome.GRASS: "res://graphics/backgrounds/background1.png",
  Biome.DESERT: "res://graphics/backgrounds/background2.png",
  Biome.ICE: "res://graphics/backgrounds/background3.png",
}
var player_monsters = [
  new_monster_res(Monster.SPARCHU, 60),
  new_monster_res(Monster.ATROX, 10),
  new_monster_res(Monster.GULFIN, 50),
  new_monster_res(Monster.CLEAF, 50),
  new_monster_res(Monster.FINSTA, 50),
  new_monster_res(Monster.PLUMETTE, 50),
]
var enemy_monsters = [
  new_monster_res(Monster.CINDRILL, 18),
  new_monster_res(Monster.POUCH, 19),
  new_monster_res(Monster.JACANA, 17),
  new_monster_res(Monster.LARVEA, 17),
]
var current_biome: Biome = Biome.DESERT
var trainer_fight: bool = false

func new_monster_res(id, level):
  var monster_res = MonsterResource.new()
  monster_res.setup(id, level)
  return monster_res
```

monster_res.gd

Found in folder: **/resources**



This code defines a `MonsterResource` class that extends the `Resource` class. It contains properties such as `id`, `level`, `current_hp`, and `current_ep`, and methods to set up the monster's stats, retrieve specific stats, get the monster's element, get available attacks based on the monster's level, and select a random attack from the available attacks based on the current EP. The code uses the `Data` singleton to access monster and attack data.

```
class_name MonsterResource extends Resource

@export var id: Data.Monster
@export var level: int
var current_hp: float
var current_ep: float

func setup(new_id, new_level):
    id = new_id
    level = new_level
    current_hp = get_stat('max hp')
    current_ep = get_stat("max ep")

func get_stat(stat: String):
    return Data.monster_data[id]['stats'][stat] * level

func get_element() -> Data.Element:
    return Data.monster_data[id]['element']

func get_attacks() -> Array:
    var attack: Array
    for level_req in Data.monster_data[id]['attacks'].keys():
        if level >= level_req:
            attack.append(Data.monster_data[id]['attacks'][level_req])
    return attack

func get_random_attack():
    var attacks = get_attacks()
    var available_attacks = []
    for attack in attacks:
        if Data.attack_data[attack]['cost'] <= current_ep:
            available_attacks.append(attack)
    return available_attacks.pick_random() if available_attacks.size() else null
```

battle.gd

Found in folder: **/scenes/battle**

This code sets up a battle scene in a game, positioning player and enemy monsters based on their starting positions. It handles monster actions like attacking and swapping, updating health and energy accordingly. The code also manages reserve monsters, allowing them to replace fallen monsters in battle.

```
extends Node2D
```

```
var battle_sprite_scene = preload("res://scenes/battle/battle_sprite.tscn")
var current_battle_sprite: Sprite2D
var player_reserve_monsters: Array
var enemy_reserve_monsters: Array

func _ready() -> void:
    $BG.texture = load(Data.biome_bg[Data.current_biome])
    battle_sprite_setup($StartPositions/Player, $BattleSprites/Player, true, Data.player_monsters) # player
    battle_sprite_setup($StartPositions/Enemy, $BattleSprites/Enemy, false, Data.enemy_monsters) # enemy

func battle_sprite_setup(start_positions: Node2D, parent: Node2D, is_player: bool, data: Array):
    for start_pos_index: int in start_positions.get_child_count():
        if start_pos_index < data.size():
            var pos = start_positions.get_child(start_pos_index).position
            var monster_res: MonsterResource = data[start_pos_index]
            create_battle_sprite(pos, parent, is_player, monster_res, start_pos_index)
    if is_player:
        player_reserve_monsters = data.slice(3)
    else:
        enemy_reserve_monsters = data.slice(3)

func create_battle_sprite(pos: Vector2, parent: Node2D, is_player: bool, res: MonsterResource, index: int):
    var battle_sprite = battle_sprite_scene.instantiate()
    battle_sprite.setup(pos, is_player, res, index)
    parent.add_child(battle_sprite)
    battle_sprite.connect("is_ready", monster_ready)
    battle_sprite.connect('death', monster_death)

func monster_ready(battle_sprite: Sprite2D):
    current_battle_sprite = battle_sprite
    if battle_sprite.playable:
        $BattleMenu.reveal(battle_sprite, player_reserve_monsters)
    else:
        var attack = battle_sprite.monster_res.get_random_attack()
        if attack is Data.Attack:
            var offensive = Data.attack_data[attack]['offensive']
            var target_group = $BattleSprites/Player if offensive else $BattleSprites/Enemy
            var target = target_group.get_children().pick_random() as Sprite2D
            $EffectSprites/BattleAttackSprite.play(attack, target.position)
            current_battle_sprite.attack_animation()

            var amount = Data.attack_data[attack]['amount'] * current_battle_sprite.monster_resource.level * 0.25
            target.change_health(amount, Data.attack_data[attack]['element'])
            current_battle_sprite.change_energy(-Data.attack_data[attack]['cost'])
            battle_continue()
```

```
func monster_death(is_player: bool, index: int):
    var reserve_list = player_reserve_monsters if is_player else enemy_reserve_monsters
    var start_positions = $StartPositions/Player if is_player else $StartPositions/Enemy
    var pos = start_positions.get_child(index).position
    var parent = $BattleSprites/Player if is_player else $BattleSprites/Enemy
    if reserve_list.size():
        create_battle_sprite(pos, parent, is_player, reserve_list.pop_at(0), index)

    if player_reserve_monsters.size() == 0 and $BattleSprites/Player.get_child_count() == 1:
        get_tree().quit()

    if enemy_reserve_monsters.size() == 0 and $BattleSprites/Enemy.get_child_count() == 1:
        finish_battle()

func _on_battle_menu_defend() -> void:
    $BattleMenu.finish()
    battle_continue()

func battle_continue():
    await get_tree().create_timer(0.5).timeout
    current_battle_sprite = null
    for battle_sprite in get_tree().get_nodes_in_group('BattleSprites'):
        battle_sprite.paused = false

func finish_battle():
    print('battle finished')

func _on_battle_menu_select(state: int, info: Variant) -> void:
    if state == Data.MenuState.ATTACK:
        var attack_data = Data.attack_data[info]
        var target = $BattleSprites/Enemy if attack_data['offensive'] else $BattleSprites/Player
        $EffectSprites/BattleSelectSprite.activate(target, info, Data.MenuState.ATTACK)
    if state == Data.MenuState.CATCH:
        var target = $BattleSprites/Enemy
        $EffectSprites/BattleSelectSprite.activate(target, info, Data.MenuState.CATCH)

func _on_battle_select_sprite_cancel(state: int) -> void:
    $BattleMenu.cancel(state)

func _on_battle_select_sprite_use(index: int, info: Variant, state: int) -> void:
    if state == Data.MenuState.ATTACK:
        var offensive = Data.attack_data[info]['offensive']
        var target_group = $BattleSprites/Enemy if offensive else $BattleSprites/Player
        var target = target_group.get_child(index) as Sprite2D
        $EffectSprites/BattleAttackSprite.play(info, target.position)
```

```
current_battle_sprite.attack_animation()

var damage = Data.attack_data[info]['amount'] * current_battle_sprite.monster_res.level * 0.255
var element = Data.attack_data[info]['element']
var cost = Data.attack_data[info]['cost']
target.change_health(damage, element)
current_battle_sprite.change_energy(-cost)
if state == Data.MenuState.CATCH:
    var target = $BattleSprites/Enemy.get_child(index)
    if target.monster_res.current_hp <= target.monster_res.get_stat('max hp') * 0.9:
        player_reserve_monsters.append(target.monster_res)

    target.queue_free()
battle_continue()

func _on_battle_menu_swap(index: int, res: MonsterResource) -> void:
    player_reserve_monsters.remove_at(index)
    player_reserve_monsters.append(current_battle_sprite.monster_res)
    current_battle_sprite.change(res)
    battle_continue()
```

battle_attack_button.gd

Found in folder: **/scenes/battle**

This code defines a custom button that displays information about an attack and emits a signal when pressed. The `setup` function sets the button's text to the attack's name and disables the button if the current EP is less than the attack's cost. When the button is pressed, it emits the `press` signal with the `ATTACK` menu state and the attack information.

```
extends Button

var info: Data.Attack
signal press(state: Data.MenuState, info: Data.Attack)

func setup(attack_enum: Data.Attack, current_ep: float):
    text = Data.attack_data[attack_enum]['name']
    info = attack_enum
    if current_ep < Data.attack_data[attack_enum]['cost']:
        disabled = true

func _on_pressed() -> void:
    press.emit(Data.MenuState.ATTACK, info)
```

battle_attack_sprite.gd

Found in folder: **/scenes/battle**

This code defines a function `play` that takes an `attack` and a `pos` parameter. It sets the position

of the sprite to `pos`, loads the texture based on the `attack` data, and creates a tween animation. The tween animation makes the sprite visible, animates the sprite frames from 0 to `hframes - 1` over 0.8 seconds, and then makes the sprite invisible again.

```
extends Sprite2D
```

```
func play(attack: Data.Attack, pos: Vector2):  
    position = pos  
    texture = load(Data.attack_data[attack]['texture'])  
    var tween = create_tween()  
    tween.tween_property(self, "visible", true, 0)  
    tween.tween_property(self, "frame", hframes - 1, 0.8).from(0)  
    tween.tween_property(self, "visible", false, 0)
```

battle_main_button.gd

Found in folder: **/scenes/battle**

This code defines a custom button class that extends the TextureButton class. It declares two variables, `state` and `info`, of type `Data.MenuState`, and a custom signal named `press` that emits the `state` and `info` values when the button is pressed. The `\_on\_pressed()` function is connected to the button's pressed signal and emits the custom `press` signal with the current `state` and `info` values.

```
extends TextureButton
```

```
var state: Data.MenuState  
var info: Data.MenuState  
signal press(state: Data.MenuState, info: Data.MenuState)
```

```
func _on_pressed() -> void:  
    press.emit(state, info)
```

battle_menu.gd

Found in folder: **/scenes/battle**

This code defines a battle menu that extends the Control node. It preloads two button scenes for battle actions (attack and swap) and sets up variables for monster resources, menu state, and a list of monsters. The script handles input, reveals the battle menu, sets up button handlers for different menu states, and emits signals for selected actions like attacking, defending, swapping monsters, or catching.

```
extends Control  
var battle_attack_button_scene = preload("res://scenes/battle/battle_attack_button.tscn")  
var battle_swap_button_scene = preload("res://scenes/battle/battle_swap_button.tscn")  
  
var monster_res: MonsterResource  
var state: Data.MenuState: set = state_handler
```



```
var monster_list: Array
const offset = 24
const button_link = {
    0: Data.MenuState.ATTACK,
    1: Data.MenuState.DEFEND,
    2: Data.MenuState.SWAP,
    3: Data.MenuState.CATCH,
}

signal defend
signal select(state: Data.MenuState, info)
signal swap(index: int, res: MonsterResource)

func _ready() -> void:
    for node in get_children():
        node.hide()

func _input(_event: InputEvent) -> void:
    if Input.is_action_just_pressed("ui_cancel") and not $Support/StopTimer.time_left:
        if state in [Data.MenuState.ATTACK, Data.MenuState.SWAP]:
            state = Data.MenuState.MAIN

func reveal(battle_sprite: Sprite2D, reserve_monsters: Array):
    show()
    state = Data.MenuState.MAIN
    position = battle_sprite.position
    monster_res = battle_sprite.monster_res
    monster_list = reserve_monsters

func state_handler(new_state):
    state = new_state
    for node in get_children():
        node.hide()
    match state:
        Data.MenuState.MAIN:
            $MainSelection.show()
            main_button_setup()
        Data.MenuState.ATTACK:
            $AttackContainer.show()
            attack_button_setup()
        Data.MenuState.CATCH:
            select.emit(Data.MenuState.CATCH, null)
        Data.MenuState.SWAP:
            $SwapContainer.show()
            swap_button_setup()

func button_handler(current_state: Data.MenuState, info):
    match current_state:
        Data.MenuState.MAIN:
            state = info
        Data.MenuState.ATTACK:
```

```
state = Data.MenuState.SELECT
select.emit(Data.MenuState.ATTACK, info)
Data.MenuState.SWAP:
    swap.emit(info[0], info[1])
    finish()
if current_state == Data.MenuState.MAIN and info == Data.MenuState.DEFEND:
    defend.emit()

func main_button_setup():
    if Data.trainer_fight:
        for i in 3:
            var button = $MainSelection.get_child(i) as TextureButton
            var angle = deg_to_rad((offset * i) - offset)
            button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)
            button.state = Data.MenuState.MAIN
            button.info = button_link[i]
            $MainSelection.get_child(-1).hide()
        else:
            for i: int in $MainSelection.get_child_count():
                var button = $MainSelection.get_child(i) as TextureButton
                var angle = deg_to_rad((offset * i) - offset * 1.5)
                button.position = Vector2(50,0).rotated(angle) + Vector2(0,-8)
                button.state = Data.MenuState.MAIN
                button.info = button_link[i]
            await get_tree().process_frame
            $MainSelection.get_child(0).grab_focus()
            for button in $MainSelection.get_children():
                if not button.is_connected('press', button_handler):
                    button.connect('press', button_handler)

func attack_button_setup():
    for button in $AttackContainer.get_children():
        button.queue_free()
    for attack_enum in monster_res.get_attacks():
        var btn = battle_attack_button_scene.instantiate()
        btn.setup(attack_enum, monster_res.current_ep)
        btn.connect('press', button_handler)
        $AttackContainer.add_child(btn)
    await get_tree().process_frame
    $AttackContainer.get_child(0).grab_focus()

func swap_button_setup():
    for btn in $SwapContainer/ScrollContainer/VBoxContainer.get_children():
        btn.queue_free()
    for i in monster_list.size():
        var btn = battle_swap_button_scene.instantiate() as Button
        var res = monster_list[i]
        btn.setup(res, i)
        btn.connect('press', button_handler)
        $SwapContainer/ScrollContainer/VBoxContainer.add_child(btn)
    await get_tree().process_frame
    $SwapContainer/ScrollContainer/VBoxContainer.get_child(0).grab_focus()
```

```
func finish():
    for node in get_children():
        node.hide()
    hide()

func cancel(new_state: Data.MenuState):
    if new_state == Data.MenuState.ATTACK:
        state = Data.MenuState.ATTACK
    if new_state == Data.MenuState.CATCH:
        state = Data.MenuState.MAIN
    $Support/StopTimer.start()
```

battle_select_sprite.gd

Found in folder: **/scenes/battle**

This code defines a Sprite2D subclass that represents a menu cursor. The cursor can be activated with a target, info, and state, and it emits signals when the user cancels or selects an option. The cursor's position is updated based on user input, and its texture is determined by the current state.

```
extends Sprite2D

var target: Node2D
var info
var state: Data.MenuState
var index: int
const textures = {
    Data.MenuState.ATTACK: preload("res://graphics/UI/options5.png"),
    Data.MenuState.CATCH: preload("res://graphics/UI/options8.png"),
}

signal cancel(state: Data.MenuState)
signal use(index: int, info, state: Data.MenuState)

func _ready() -> void:
    hide()

func activate(new_target, new_info, new_state: Data.MenuState):
    show()
    target = new_target
    info = new_info
    state = new_state
    index = 0
    position = target.get_child(index).position
    texture = textures[state]

func _input(_event: InputEvent) -> void:
    if visible:
        if Input.is_action_just_pressed("ui_down") or Input.is_action_just_pressed("ui_up")
        :
```



```
var dir = Input.get_axis("ui_up", "ui_down")
index = posmod(index + dir, target.get_child_count())
position = target.get_child(index).position
if Input.is_action_just_pressed("ui_cancel"):
    hide()
    cancel.emit(state)
if Input.is_action_just_pressed("ui_accept"):
    use.emit(index, info, state)
    hide()
```

battle_sprite.gd

Found in folder: **/scenes/battle**

This code defines a Sprite2D subclass representing a monster in a battle system. It handles the monster's readiness, health, and energy, and emits signals when the monster is ready to act or dies. The class also includes methods for setting up the monster's UI, updating its health and energy, and playing attack animations.

```
extends Sprite2D
var monster_res: MonsterResource
var paused: bool
var readiness: float
var playable: bool
var index: int

signal is_ready(battle_sprite: Sprite2D)
signal death(is_player: bool, index: int)

func _process(delta: float) -> void:
    if not paused:
        readiness += monster_res.get_stat('speed') * delta
        $Control/ReadyProgressBar.value = readiness
        change_energy(monster_res.get_stat('max ep') * delta * 0.01)

func setup(pos: Vector2, is_player: bool, res: MonsterResource, new_index: int):
    position = pos
    flip_h = is_player
    playable = is_player
    monster_res = res
    index = new_index
    texture = load(Data.monster_data[res.id]['battle texture'])
    ui_setup()

func ui_setup():
    # name
    $Control/NameContainer/MarginContainer/Label.text = Data.monster_data[monster_res.id]
    ['name']
    # health
    $Control/StatsContainers/HealthBar.max_value = monster_res.get_stat("max hp")
    $Control/StatsContainers/HealthBar.value = monster_res.current_hp
```

```
# energy
$Control/StatsContainers/Control/EnergyBar.max_value = monster_res.get_stat('max ep'
)
$Control/StatsContainers/Control/EnergyBar.value = monster_res.current_ep

func _on_ready_progress_bar_value_changed(value: float) -> void:
    if value >= 100:
        for battle_sprite in get_tree().get_nodes_in_group("BattleSprites"):
            battle_sprite.paused = true
        readiness = 0
        is_ready.emit(self)

func attack_animation():
    $AnimationPlayer.play('attack')
    await $AnimationPlayer.animation_finished
    $AnimationPlayer.play('idle')

func change_health(amount: float, element: Data.Element):
    var modifier = Data.element_modifier[element][monster_res.get_element()]
    monster_res.current_hp -= amount * modifier
    var tween = create_tween()
    tween.tween_property(
        $Control/StatsContainers/HealthBar,
        "value",
        monster_res.current_hp,
        0.5)
    tween.tween_callback(check_death)

func change_energy(amount: float):
    monster_res.current_ep += amount
    monster_res.current_ep = clamp(monster_res.current_ep, 0, monster_res.get_stat("max ep"))
    var tween = create_tween()
    tween.tween_property(
        $Control/StatsContainers/Control/EnergyBar,
        "value",
        monster_res.current_ep,
        0.5)

func check_death():
    if monster_res.current_hp <= 0:
        death.emit(playable, index)
        queue_free()

func change(res: MonsterResource):
    texture = load(Data.monster_data[res.id]['battle texture'])
    monster_res = res
    ui_setup()
```

battle_swap_button.gd

Found in folder: **/scenes/battle**

This code defines a custom button that displays information about a monster. When the button is pressed, it emits a signal with the monster's index and resource. The button is set up with a monster resource and index, and displays the monster's name and icon texture.

```
extends Button
```

```
var res: MonsterResource
```

```
var index: int
```

```
signal press(state: Data.MenuState, info)
```

```
func setup(monster_res: MonsterResource, new_index: int):
```

```
    res = monster_res
```

```
    index = new_index
```

```
    $HBoxContainer/Label.text = Data.monster_data[res.id]['name']
```

```
    $HBoxContainer/TextureRect.texture = load(Data.monster_data[res.id]['icon texture'])
```

```
func _on_pressed() -> void:
```

```
    press.emit(Data.MenuState.SWAP, [index, res])
```

character.gd

Found in folder: **/scenes/characters**

This code defines a Character class that extends the CharacterBody2D class. It has properties for character style, view distance, direction, speed, current horizontal frame, view direction, and a flag for whether the character can move. The code also includes functions for moving the character, animating the character's sprite based on its direction and view, getting the direction towards a target character, and changing the character's view direction.

```
class_name Character extends CharacterBody2D
```

```
@export var char_style: Data.CharacterStyle
```

```
@export var view_distance := 50
```

```
var direction: Vector2
```

```
var speed = 60
```

```
var current_h_frame: float
```

```
var view_direction: Vector2i
```

```
var can_move := true
```

```
func _ready() -> void:
```

```
    $Sprite2D.texture = load(Data.character_texture_data[char_style])
```

```
func move():
```

```
    velocity = direction * speed
```

```
    move_and_slide()
```

```
func animate(delta):
    if direction:
        var face_dir: Vector2i = Vector2i(round(direction.x), round(direction.y))
        $Sprite2D.frame_coords.y = Data.character_view_directions[face_dir]
        current_h_frame += Data.ANIMATION_SPEED * delta
    else:
        current_h_frame = 0
    $Sprite2D.frame_coords.x = int(current_h_frame) % $Sprite2D.hframes
```

```
func get_char_direction(target_char) -> Vector2i:
    if target_char and self:
        var dir = (target_char.position - position).normalized()
        return Vector2i(round(dir.x), round(dir.y))
    else:
        return Vector2i.DOWN
```

```
func change_view(target: Vector2i):
    $Sprite2D.frame_coords.y = Data.character_view_directions[target]
```

dialog_box.gd

Found in folder: **/scenes/characters**

This code defines a custom Godot node that extends the PanelContainer class. It includes a `set_text` function that takes a `String` parameter named `text`. When called, this function sets the text property of a Label node located inside a MarginContainer within the scene hierarchy.

```
extends PanelContainer

func set_text(text: String):
    $MarginContainer/Label.text = text
```

player.gd

Found in folder: **/scenes/characters**

This code defines a class that extends the Character class. It handles player input, movement, animation, and interaction with other characters. The code also includes functions for updating the player's view direction, checking for interactable objects, and stopping movement.

```
extends Character

var interactable: Character

func _process(delta: float) -> void:
    get_input()
    if can_move:
        move()
```

```
animate(delta)
update_view_direction()
check_interactable()

func get_input():
    if not Data.current_char:
        direction = Input.get_vector("left", "right", "up", "down")
        if Input.is_action_just_pressed("action") and interactable:
            Data.current_char = interactable
            Data.current_char.change_view(-get_char_direction(Data.current_char))
            Data.current_char.look_around = false
            Data.current_char.show_dialog()
        else:
            direction = Vector2.ZERO
            change_view(get_char_direction(Data.current_char))
            if Input.is_action_just_pressed("action"):
                Data.current_char.advance_dialog()

    if Input.is_action_just_pressed("ui_focus_next"):
        for trainer in get_tree().get_nodes_in_group('Trainers'):
            trainer.test_defeat()

func update_view_direction():
    if direction:
        var y = round(direction.y) if direction.x == 0 else 0
        view_direction = Vector2i(round(direction.x),y)
        $RayCast2D.target_position = view_direction * view_distance

func check_interactable():
    if $RayCast2D.collide_with_bodies:
        interactable = $RayCast2D.get_collider()

func stop():
    direction = Vector2.ZERO
    can_move = false
```

trainer.gd

Found in folder: **/scenes/characters**

This code defines a character in a game that can interact with the player. The character has properties such as dialog, defeat dialog, and a unique ID. The character can detect when the player is nearby, turn to face the player, and display dialog when the player is within a certain distance.

```
extends Character
```

```
@onready var player = get_tree().get_first_node_in_group('Player')
@export var stop_radius := 24
@export var look_around := true
```

```
@export var dialog: Array[String]
@export var defeat_dialog: Array[String]
var dialog_index: int
var defeated := false:
    set(value):
        defeated = value
        Data.char_data[unique_id]['defeated'] = value
var unique_id: String

func test_defeat():
    defeated = true

func get_player():
    return get_tree().get_first_node_in_group('Player')

func _enter_tree() -> void:
    unique_id = get_unique_id()
    if unique_id not in Data.char_data:
        Data.char_data[unique_id] = {'defeated': defeated}
    else:
        defeated = Data.char_data[unique_id]['defeated']

func get_unique_id() -> String:
    var current_scene_name = get_owner().scene_file_path.get_file().get_basename()
    var node_name = name
    return current_scene_name + "_" + node_name

func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == get_player() and not Data.current_char and look_around:
        Data.current_char = self
        look_around = false
        $Timers/WalkWaitTimer.start()
    if direction:
        animate(delta)
        if position.distance_to(get_player().position) < stop_radius:
            direction = Vector2.ZERO
            $Sprite2D.frame_coords.x = 0
            show_dialog()
            move()

func _on_watch_timer_timeout() -> void:
    if look_around:
        view_direction = [Vector2i.LEFT, Vector2i.RIGHT, Vector2i.UP, Vector2i.DOWN].pick_random()
        $Sprite2D.frame_coords.y = Data.character_view_directions[view_direction]
        $RayCast2D.target_position = view_direction * view_distance
```

```
func _on_walk_wait_timer_timeout() -> void:
    direction = get_char_direction(get_player())

func show_dialog():
    $DialogBox.set_text(dialog[0] if not defeated else defeat_dialog[0])
    $DialogBox.show()

func advance_dialog():
    var current_dialog = dialog if not defeated else defeat_dialog
    if dialog_index < current_dialog.size() - 1:
        dialog_index += 1
        $DialogBox.set_text(current_dialog[dialog_index])
    else:
        $DialogBox.hide()
        print('start battle')
        Data.current_char = null
        dialog_index = 0
```

level.gd

Found in folder: **/scenes/levels**

This code sets up tileset animations for the floor and water tiles, creates collision shapes for simple objects like trees, and changes the background color to black. It also positions the player at the start of a transition gate based on the target location and adjusts the player's view direction accordingly.

```
extends Node2D
```

```
func _ready() -> void:
    # tileset animation
    var tileset = $BG/Floor.tile_set as TileSet
    var atlas_source = tileset.get_source(2) as TileSetAtlasSource
    for y in atlas_source.get_atlas_grid_size().y:
        for x in int(atlas_source.get_atlas_grid_size().x * 0.25):
            var pos = Vector2i(x * 4, y)
            atlas_source.set_tile_animation_frames_count(pos, 4)
            atlas_source.set_tile_animation_speed(pos, Data.TILE_ANIMATION_SPEED)
    var water_source = tileset.get_source(1)
    water_source.set_tile_animation_speed(Vector2.ZERO, Data.TILE_ANIMATION_SPEED)

    # sprite collisions
    for sprite: Sprite2D in $Objects/SimpleObjects.get_children():
        var texture = sprite.texture
        var shape = RectangleShape2D.new()
        shape.size = texture.get_size()
        shape.size.y *= 0.7
        if 'Tree' in sprite.name:
            shape.size.x *= 0.4
        var collision_shape = CollisionShape2D.new()
        collision_shape.shape = shape
        var static_body = StaticBody2D.new()
```



```
static_body.add_child(collision_shape)
sprite.add_child(static_body)

# change bg color
RenderingServer.set_default_clear_color(Color.BLACK)

func player_start_position(target: Data.Location):
    for gate in $TransitionGates.get_children():
        if gate.target == target:
            $Objects/Characters/Player.position = gate.position + gate.get_start_offset()
            var view_dir = gate.get_start_offset().normalized() as Vector2i
            $Objects/Characters/Player.change_view(view_dir)
```

transition_gate.gd

Found in folder: **/scenes/levels**

This code defines an Area2D node that triggers a scene transition when a player character enters it. It has exported variables for specifying the target and current locations, the starting offset direction, and the distance. When the player enters the area, the `\_on\_body\_entered` function is called, stopping the player and initiating the transition using the `TransitionLayer` singleton.

```
extends Area2D

@export var target: Data.Location
@export var current: Data.Location
@export_enum("Up", "Down", "Left", "Right") var start_offset = "Up"
@export var distance := 30

func _on_body_entered(player: Character) -> void:
    player.stop()
    TransitionLayer.transition(target, current)

func get_start_offset() -> Vector2:
    return {
        "Up": Vector2.UP,
        "Down": Vector2.DOWN,
        "Right": Vector2.RIGHT,
        "Left": Vector2.LEFT
    }[start_offset] * distance
```

transition_layer.gd

Found in folder: **/scenes/levels**

This code defines a scene transition effect using a `CanvasLayer` and a `ColorRect` node. The `transition` function animates the `ColorRect`'s opacity to create a fade-in effect, changes the scene using the `\_change\_scene` function, and then fades out the `ColorRect`. The `\_change\_scene` function loads the new scene based on the target location, sets it as the current scene, and positions



the player based on the current location.

```
extends CanvasLayer
```

```
func _ready() -> void:  
    $ColorRect.modulate.a = 0.0
```

```
func transition(target_loc: Data.Location, current_loc: Data.Location):  
    var tween = create_tween()  
    tween.tween_property($ColorRect, "modulate:a", 1.0, 0.8)  
    tween.tween_interval(0.5)  
    tween.tween_callback(_change_scene.bind(target_loc, current_loc))  
    tween.tween_property($ColorRect, "modulate:a", 0.0, 0.8)
```

```
func _change_scene(target_loc: Data.Location, current_loc: Data.Location):  
    if get_tree().current_scene:  
        get_tree().current_scene.queue_free()  
    var scene = load(Data.LEVEL_PATHS[target_loc]).instantiate()  
    get_tree().root.add_child(scene)  
    get_tree().current_scene = scene  
    scene.player_start_position(current_loc)  
    Data.current_loc = target_loc
```